Leitura — Compose: listas, formulários e Material 3 (14/09)

Guia de apoio para a aula da Sprint 1. Cobre os conceitos apresentados construindo, do zero, uma tela de tarefas em Compose Multiplatform, no Android Studio. A ideia é ler antes da aula para chegar com os termos na ponta da língua e voltar a ele durante o projeto, quando surgir uma dúvida.

Exemplo de referência (Android + Desktop): `exemplos/tarefas-compose/`, com `PASSOS.md`. Versões usadas: Kotlin 2.4.10, Compose Multiplatform 1.12.0, Android Gradle Plugin 9.1.0, compileSdk e targetSdk 37, minSdk 24.

Os comportamentos descritos aqui foram conferidos em execução, em três frentes:

- testes de interface do Compose rodando no alvo Desktop (clicar, digitar, contar recomposições);
- build do alvo Android (APK de debug) e do lint do Android;
- testes com Robolectric no alvo Android para a restauração de estado.

Em outras versões, detalhes podem mudar.

Como ler: os capítulos seguem a ordem da aula, mas cada um se sustenta sozinho. Quem já viu a Sprint 0 com atenção pode ir direto ao capítulo 4 (estado) e ao 5 (imutabilidade), que concentram as dúvidas mais comuns. Os quadros "Erro comum" reúnem o que costuma travar os grupos.

Capítulos:

1. O modelo mental do Compose
2. Ambiente: Android Studio e projeto Kotlin Multiplatform
3. Kotlin para quem sabe Java
4. Estado: `mutableStateOf`, `remember`, `rememberSaveable` e estado elevado
5. Imutabilidade: `val`, `copy` e reatribuição
6. Componentes e `Modifier`
7. Listas: `LazyColumn`, `items` e `key`
8. Formulários e validação
9. Material 3 e tema
10. `@Preview` a fundo
11. MUSI, próxima aula, exercícios e dúvidas

---

1. O modelo mental do Compose

Compose muda a pergunta que se faz ao construir uma tela. Em vez de "que widget preciso alterar quando isto acontecer?", a pergunta passa a ser "como a tela deve ficar dado este estado?". Este capítulo explica essa troca e as regras que decorrem dela.

1.1 O jeito imperativo: árvore de widgets e setters

No sistema de Views clássico do Android, a tela é uma árvore de objetos, normalmente inflada de um XML. Cada widget guarda estado interno e expõe getters e setters. Para mudar a tela, o código percorre a árvore e chama setters:

```
// Views clássicas (Java): o código mantém a tela sincronizada à mão
TextView contador = findViewById(R.id.contador);
Button botao = findViewById(R.id.botao);

botao.setOnClickListener(v -> {
  cliques++;
  contador.setText("Cliquei " + cliques);   // lembrar de atualizar CADA lugar que mostra 'cliques'
  if (cliques >= 10) botao.setEnabled(false);
});
```

A documentação do Compose aponta os problemas desse modelo: se o mesmo dado aparece em vários lugares, é fácil esquecer de atualizar um deles; duas atualizações podem entrar em conflito (por exemplo, alterar um widget que acabou de ser removido); e a complexidade cresce com o número de widgets que precisam ser mantidos em dia.

1.2 O jeito declarativo: a tela é função do estado

Em Compose, você escreve funções que recebem dados e descrevem a interface:

```
@Composable
fun Contador(cliques: Int, onClique: () -> Unit) {
  Button(onClick = onClique, enabled = cliques < 10) {
    Text("Cliquei $cliques")
  }
}
```

Não há `setText` nem `setEnabled`. O texto e a habilitação são calculados a partir de `cliques` toda vez que a função roda. Quando `cliques` muda, o Compose chama a função de novo e aplica na tela só o que ficou diferente. A frase que resume o modelo:

```
UI = f(estado)
```

As características de uma função `@Composable`, segundo a documentação:

Característica	No exemplo
anotada com `@Composable`	avisa o compilador do Compose que a função descreve UI

Característica	No exemplo
recebe dados como parâmetros	`cliques`, `onClique`
emite UI chamando outras composables	`Button`, `Text`
não devolve nada	descreve a tela; não constrói um objeto de widget
rápida, idempotente e sem efeitos colaterais	mesmos argumentos, mesma descrição

Kotlin para quem conhece Java: não existe um objeto `Contador` que você guarde numa variável e altere depois. A "tela" é o resultado de chamar a função; para mudá-la, a função é chamada de novo com outros argumentos.

📖 Ref. Android Developers — Thinking in Compose: https://developer.android.com/develop/ui/compose/mental-model

1.3 Composição e recomposição

Três termos aparecem o tempo todo:

- Composição: a descrição da interface que o Compose monta ao executar as composables.
- Composição inicial: a primeira execução, que cria essa descrição.
- Recomposição: executar de novo as composables afetadas quando um estado lido por elas muda, atualizando a descrição.

O que dispara a recomposição é a mudança de um estado observável (capítulo 4) que foi lido durante a composição. O Compose registra quem leu cada estado e, quando o valor muda, reexecuta só esses trechos. Funções cujos parâmetros não mudaram são puladas.

Isso foi medido com a lista do exemplo. Três cartões, contando quantas vezes cada composable executou:

```
composição inicial:      pai=1, cartao-1=1, cartao-2=1, cartao-3=1
depois de alternar B:    cartao-2=1
```

Ao marcar a tarefa B, só o cartão B recompôs. O pai, que guarda a lista, e os outros dois cartões foram pulados. Não é preciso otimizar nada para obter esse comportamento; ele vem de a lista ser imutável (capítulo 5) e de os cartões receberem só o que mostram.

Um detalhe que explica comportamentos estranhos mais adiante (seção 4.2): a unidade de recomposição não é "a função inteira", e sim o trecho que leu o estado. Um estado lido só dentro do conteúdo de um `Button { ... }` pode recompor apenas esse conteúdo, sem reexecutar a função que contém o botão.

1.4 Regras que decorrem do modelo

Como o Compose decide quando e quantas vezes executar cada função, a documentação lista propriedades que o seu código precisa tolerar:

- Recomposição pula o máximo possível. Não conte que uma função vai executar.

- Recomposição é otimista e pode ser cancelada, se o estado mudar de novo no meio dela.
- Uma composable pode executar com muita frequência, até a cada quadro de uma animação.
- Composables podem executar em qualquer ordem.
- Composables podem vir a executar em paralelo (hoje não executam, mas o código deve ser escrito como se pudessem).

Consequência prática: o corpo de uma composable não deve ter efeitos colaterais. Exemplos do que não fazer no corpo da função:

```
@Composable
fun ListaComDefeito(itens: List<String>) {
  var contagem = 0
  Column {
    for (item in itens) {
      Text(item)
      contagem++                // efeito colateral: depende de quantas vezes o trecho rodou
    }
  }
  Text("Total: $contagem")      // pode mostrar um número errado
  salvarNoDisco(itens)          // pode rodar dezenas de vezes, ou nenhuma
}
```

Onde colocar efeitos: nos callbacks de evento (`onClick` , `onValueChange`), que rodam uma vez por interação, ou nas APIs de efeito do Compose (`LaunchedEffect` e afins), assunto de outra aula.

📖 Ref. Android Developers — Recomposition (em Thinking in Compose):
https://developer.android.com/develop/ui/compose/mental-model#recomposition

1.5 As três fases de um quadro

Para desenhar um quadro, o Compose passa por três fases:

```
estado ──► Composição ──► Layout ──► Desenho ──► tela
           o que mostrar   onde       como
                           (medir e   (pintar no
                           posicionar) canvas)
```

1. Composição: executa as composables e produz a árvore que descreve a UI.
2. Layout: mede e posiciona cada nó. Cada nó mede os filhos, decide o próprio tamanho e posiciona os filhos, numa única passada pela árvore.
3. Desenho: cada nó se pinta, de cima para baixo.

Um estado lido numa fase só reexecuta aquela fase em diante. Ler um estado dentro de um modificador de desenho, por exemplo, pode evitar a composição e o layout. Isso importa para animações e otimização; nesta aula, basta saber que composição é a primeira fase e é onde o seu código `@Composable` roda. `LazyColumn` é

uma exceção notável: a composição dos itens depende do layout (quais itens cabem na tela), assunto do capítulo 7.

📖 Ref. Android Developers — Jetpack Compose phases:
https://developer.android.com/develop/ui/compose/phases

1.6 Estado desce, eventos sobem

O desenho que o exemplo inteiro segue:

```
              App()  ── guarda: tarefas, texto
             /     \
   estado ↓ /       \ ↓ estado
           /         \
FormularioTarefa    CartaoTarefa (um por tarefa)
     |  ↑                |  ↑
     |  └ onAdicionar    |  └ onAlternar
     |  └ onTextoChange  |
   evento ↑           evento ↑
```

Os componentes de baixo recebem dados e avisam o que aconteceu; quem guarda o estado decide o que fazer e altera o estado; a recomposição leva o novo estado para baixo. É o fluxo de dados unidirecional, detalhado no capítulo 4.

1.7 Erros comuns do modelo mental

> Erro comum: esperar que o campo de texto se atualize sozinho. `OutlinedTextField(value = "", onValueChange = { })` não muda quando o usuário digita. No teste, depois de digitar "oi", o campo continuou vazio. O campo mostra `value`; para ele mudar, `onValueChange` precisa atualizar um estado que alimenta `value` (seção 4.2).

> Erro comum: efeito colateral no corpo da composable. Contador incrementado, chamada de rede, gravação em arquivo ou log com lógica, tudo no corpo da função. Esses efeitos rodam quantas vezes o Compose decidir. Mova para callbacks de evento ou para APIs de efeito.

> Erro comum: pensar em "atualizar a tela". Não existe um `refresh()`. Se a tela não mudou, a pergunta certa é: qual estado deveria ter mudado, e ele é observável? (capítulos 4 e 5)

---

2. Ambiente: Android Studio e projeto Kotlin Multiplatform

2.1 Por que Android Studio

A aula usa o Android Studio instalado no laboratório. Duas ferramentas do dia a dia dependem do ambiente local:

- `@Preview`: renderiza composables dentro da IDE, sem emulador (seção 2.6 e capítulo 10).
- Compose Hot Reload: aplica mudanças de código numa janela Desktop já aberta (seção 2.5).

No Codespaces, as duas funcionam mal, como visto na Sprint 0. O emulador Android continua disponível para ver o app no formato de telefone; ele é criado no Device Manager do Android Studio.

📖 Ref. Android Studio — Run apps on the Android Emulator: https://developer.android.com/studio/run/emulator

2.2 Kotlin Multiplatform em uma página

Kotlin Multiplatform (KMP) permite compilar o mesmo código Kotlin para plataformas diferentes. Compose Multiplatform leva a interface junto: a mesma função `App()` roda no Android, no Desktop (JVM), no iOS e na web. O Compose Multiplatform é construído pela JetBrains sobre o Jetpack Compose do Google e compartilha com ele o compilador e o runtime do Compose; as APIs de interface (`Column`, `Text`, `remember`) são as mesmas.

O código fica organizado em source sets, grupos de arquivos com dependências próprias:

Source set	O que contém	Compila para
`commonMain`	código comum: a UI e os modelos	todos os alvos
`androidMain`	código só do Android (`MainActivity`)	Android
`desktopMain`	código só do Desktop (`main()`)	JVM

A regra: escreva em `commonMain` sempre que possível; deixe nos source sets de plataforma apenas o que usa API específica. Quando o código comum precisa de algo que cada plataforma implementa de um jeito, o Kotlin oferece `expect` e `actual`:

```
// commonMain
expect fun nomeDaPlataforma(): String

// androidMain
actual fun nomeDaPlataforma(): String = "Android ${android.os.Build.VERSION.SDK_INT}"

// desktopMain
actual fun nomeDaPlataforma(): String = "Desktop (${System.getProperty("os.name")})"
```

O compilador verifica que cada `expect` tem um `actual` em cada alvo. (Esse trecho não está no exemplo; foi compilado à parte, para os dois alvos.)

📖 Ref. Kotlin Multiplatform — Project structure: https://kotlinlang.org/docs/multiplatform/multiplatform-discover-project.html

📖 Ref. Kotlin Multiplatform — Expected and actual declarations: https://kotlinlang.org/docs/multiplatform/multiplatform-expect-actual.html

2.3 O projeto `tarefas-compose`

```
tarefas-compose/
├── settings.gradle.kts            repositórios e o módulo :composeApp
├── build.gradle.kts               declara os plugins (aplicados no módulo)
├── gradle.properties              flags do AGP 9
├── gradle/libs.versions.toml      catálogo de versões
├── local.properties               caminho do Android SDK (por máquina, fora do git)
└── composeApp/
    ├── build.gradle.kts           alvos, dependências por source set, config Android e Desktop
    └── src/
        ├── commonMain/kotlin/.../App.kt           Tarefa, App, CartaoTarefa, FormularioTarefa,
previews
        ├── androidMain/kotlin/.../MainActivity.kt ponto de entrada Android
        ├── androidMain/AndroidManifest.xml
        └── desktopMain/kotlin/.../main.kt         ponto de entrada Desktop
```

O trecho do `composeApp/build.gradle.kts` que define os alvos e as dependências:

```
kotlin {
  androidTarget { /* ... */ }      // alvo Android
  jvm("desktop")                   // alvo Desktop, chamado "desktop"; daí o nome desktopMain

  sourceSets {
    commonMain.dependencies {
      implementation(compose.runtime)
      implementation(compose.foundation)
      implementation(compose.material3)
      implementation(compose.ui)
      implementation("org.jetbrains.compose.ui:ui-tooling-preview:1.12.0")   // @Preview
    }
    androidMain.dependencies {
      implementation("org.jetbrains.compose.ui:ui-tooling:1.12.0")           // render do Preview
      implementation(libs.androidx.activity.compose)                          // setContent { }
    }
    desktopMain.dependencies {
      implementation(compose.desktop.currentOs)
    }
  }
}
```

(As versões aparecem aqui literais para facilitar a leitura; no arquivo, vêm do catálogo.)

Os dois alvos compilam com essas versões: o APK de debug do Android foi gerado, e os testes do Desktop rodaram. O build emite três avisos que valem conhecer:

```
WARNING: The option setting 'android.builtInKotlin=false' is deprecated.
WARNING: The option setting 'android.newDsl=false' is deprecated.
w: The 'org.jetbrains.kotlin.multiplatform' plugin deprecated compatibility with
   Android Gradle plugin: 'com.android.application'
```

Eles dizem que a combinação usada (um único módulo KMP com o plugin de aplicação Android) está em processo de descontinuação no AGP 9. Funciona hoje; os projetos novos já nascem com outra estrutura (próxima seção).

2.4 Estrutura do assistente atual × estrutura do exemplo

Se o grupo criar um projeto novo pelo assistente de Kotlin Multiplatform, o resultado não será idêntico ao exemplo. A documentação atual descreve módulos separados:

	Exemplo da aula	Assistente atual (documentação)
Módulos	um: `composeApp`	`shared` (código comum) + `androidApp`, `desktopApp`, `iosApp`, `webApp`
Source set do Desktop	`desktopMain` (alvo `jvm("desktop")`)	`jvmMain`
Plugin Android no módulo KMP	`com.android.application`	biblioteca KMP para Android no módulo `shared`
Configuração de run Android	`composeApp`	`androidApp`

Os conceitos (source sets, `commonMain`, pontos de entrada chamando `App()`) são os mesmos. Ao seguir um tutorial, traduza os nomes: onde ele diz `jvmMain`, no exemplo é `desktopMain`.

A documentação também avisa que a IDE pode sugerir atualizar o Android Gradle Plugin e que nem toda versão nova é compatível com Kotlin Multiplatform; confira a tabela de compatibilidade antes de aceitar.

📖 Ref. Compose Multiplatform — Create your app (estrutura do projeto): https://kotlinlang.org/docs/multiplatform/compose-multiplatform-create-first-app.html

2.5 Pontos de entrada e como rodar

Cada plataforma tem um ponto de entrada, e todos fazem a mesma coisa: montar `App()`.

```
// androidMain: a Activity que o sistema abre
class MainActivity : ComponentActivity() {
  override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContent { App() }
  }
}
```

```
// desktopMain: uma janela JVM
fun main() = application {
  Window(onCloseRequest = ::exitApplication, title = "Tarefas") {
    App()
  }
}
```

Kotlin para quem conhece Java: `fun main() = application { ... }` é uma função de nível superior com corpo de expressão; `application` recebe uma lambda (seção 3.2). `::exitApplication` é uma referência de função, como `this::exitApplication` em Java.

Formas de rodar:

Alvo	Como	Observação
Android	run configuration `composeApp`, escolher o emulador, Run	ciclo mais lento: compila e instala o APK
Desktop	`./gradlew :composeApp:run`	abre uma janela; bom para testar a lógica da UI
Desktop com Hot Reload	`./gradlew :composeApp:hotRunDesktop --mainClass br.ufrn.exemplo.tarefas.MainKt`, ou a configuração com Hot Reload na IDE	aplica mudanças sem fechar a janela

A partir do Compose Multiplatform 1.10, o plugin do Hot Reload vem embutido em projetos com alvo Desktop; no exemplo, as tarefas `hotRunDesktop` e `hotReloadDesktopMain` já existem. Requisito indicado pela documentação: o Hot Reload roda sobre o JetBrains Runtime, que hoje suporta Java 21.

📖 Ref. Compose Multiplatform — Compose Hot Reload: https://kotlinlang.org/docs/multiplatform/compose-hot-reload.html

2.6 `@Preview` no projeto KMP

Uma função `@Composable` sem parâmetros, anotada com `@Preview`, é renderizada no painel Split/Design do Android Studio:

```
import androidx.compose.ui.tooling.preview.Preview

@Preview(showBackground = true)
@Composable
fun CartaoTarefaPreview() {
  CartaoTarefa(tarefa = Tarefa(1, "Estudar Compose", feita = true), onAlternar = {})
}
```

Três fatos do projeto KMP:

- O Preview em `commonMain` precisa de um alvo Android: a renderização usa bibliotecas do Android.
- Existem duas anotações. A atual é `androidx.compose.ui.tooling.preview.Preview` (dependência `org.jetbrains.compose.ui:ui-tooling-preview`), multiplataforma desde o Compose Multiplatform 1.10. A antiga, `org.jetbrains.compose.ui.tooling.preview.Preview`, está descontinuada e exige outra dependência.
- A renderização no Android Studio precisa de `org.jetbrains.compose.ui:ui-tooling` no classpath Android. O exemplo a declara em `androidMain`; a documentação sugere `debugImplementation`, para não ir ao APK de release.

Limitações dos previews, segundo a documentação do Android: sem acesso a rede nem a arquivos, e dificuldade com `ViewModel` (o preview não sabe construir as dependências). É mais um motivo para componentes receberem dados por parâmetro (capítulo 4): um componente assim é pré-visualizável com dados fixos. O capítulo 10 aprofunda parâmetros, múltiplos previews e o modo interativo.

📖 Ref. Compose Multiplatform — Compose UI previews: https://kotlinlang.org/docs/multiplatform/compose-previews.html

📖 Ref. Android Developers — Preview your UI: https://developer.android.com/develop/ui/compose/tooling/previews

2.7 Erros comuns de ambiente

> Erro comum: importar a anotação antiga do Preview. `import org.jetbrains.compose.ui.tooling.preview.Preview` no exemplo não compila: `Unresolved reference 'compose'`, porque a dependência declarada é a da anotação nova. Use `import androidx.compose.ui.tooling.preview.Preview`.

> Erro comum: seguir um tutorial com `jvmMain` e criar a pasta errada. No exemplo, o alvo Desktop se chama `desktop`, e o source set é `desktopMain`. Um arquivo em `src/jvmMain` não faz parte de nenhum source set e é ignorado.

> Erro comum: código de plataforma em `commonMain`. `android.os.Build`, `java.io.File` ou `Activity` em `commonMain` não compilam para todos os alvos. Mova para o source set da plataforma, ou use `expect` / `actual`.

> Erro comum: `local.properties` de outra máquina. O arquivo guarda o caminho do Android SDK daquela máquina e fica fora do git. Se o Gradle não encontra o SDK, abra o projeto no Android Studio (que recria o arquivo) ou ajuste `sdk.dir`.

> Erro comum: aceitar toda sugestão de atualização de versão. O lint lista versões mais novas de AGP, Kotlin e bibliotecas. Em projeto KMP, as versões de Kotlin, Compose Multiplatform e AGP precisam ser compatíveis entre si; atualize uma de cada vez, conferindo a tabela de compatibilidade.

---

3. Kotlin para quem sabe Java

Compose é uma API Kotlin que usa intensamente recursos que Java não tem. Este capítulo cobre o que aparece no código da aula, sempre com o paralelo em Java e o erro que costuma aparecer.

3.1 Lambdas, tipos de função e `it`

Quase todo `{ }` em código Compose é uma lambda: um bloco de código passado como valor.

```
val saudar: () -> Unit = { println("oi") }
val dobro: (Int) -> Int = { x -> x * 2 }
val dobroCurto: (Int) -> Int = { it * 2 }   // parâmetro único: pode usar "it"
```

O tipo de uma função se escreve `(Entradas) -> Saída`. Os que aparecem no exemplo:

Tipo	Onde aparece	Parecido em Java
`() -> Unit`	`onAlternar`, `onAdicionar`, `onClick`	`Runnable`
`(String) -> Unit`	`onTextoChange`, `onValueChange`	`Consumer<String>`
`(Boolean) -> Unit`	`onCheckedChange` do `Checkbox`	`Consumer<Boolean>`
`@Composable () -> Unit`	conteúdo de `Button`, `Card`, `label`	não há equivalente

`Unit` é o "nada" do Kotlin, correspondente ao `void`. `it` é o nome automático do único parâmetro de uma lambda: em `onValueChange = { texto = it }`, `it` é o texto novo.

O último tipo da tabela merece atenção. `@Composable () -> Unit` é uma lambda que emite UI; só pode ser chamada dentro de outra composable. É assim que um `Button` recebe o que desenhar dentro dele.

Kotlin para quem conhece Java: em Java, uma lambda só existe como implementação de uma interface funcional (`Runnable`, `Consumer`...). Em Kotlin, funções têm tipo próprio, sem interface intermediária.

> Erro comum: confundir parâmetro de lambda com `it` aninhado. No exemplo, `items(tarefas) { tarefa -> ... tarefas.map { it.copy(...) } }` tem dois nomes: `tarefa` é a linha da lista; `it`, dentro do `map`, é cada item percorrido. Dar nome explícito ao parâmetro externo (`tarefa ->`) evita confundir os dois.

📖 Ref. Kotlin — Higher-order functions and lambdas: https://kotlinlang.org/docs/lambdas.html

3.2 Trailing lambda e slots

Quando o último parâmetro de uma função é uma função, a lambda pode ir para fora dos parênteses. Se for o único argumento, os parênteses somem:

```
Column { /* ... */ }                       // Column(content = { ... })
Card(Modifier.fillMaxWidth()) { /* ... */ } // Card(modifier = ..., content = { ... })
Button(onClick = { contador++ }) {         // onClick: lambda de evento, entre parênteses
  Text("Cliquei $contador")                // content: lambda de UI, fora dos parênteses
}
```

É isso que dá ao Compose a aparência de linguagem própria: `Column`, `Card` e `Button` são funções comuns recebendo uma lambda como último argumento.

O `Button` mostra um padrão chamado slot: o componente não decide o que vai dentro dele; recebe uma lambda `@Composable` e a chama no lugar certo. Por isso o mesmo `Button` aceita texto, ícone ou os dois.

Kotlin para quem conhece Java: `Button(onClick = { ... }) { Text(...) }` equivale a `Button(() -> ..., () -> Text(...))`, com a segunda lambda escrita fora da chamada.

> Erro comum: trocar a lambda de evento pela de conteúdo. `Button({ Text("Salvar") }) { salvar() }` não compila: o primeiro parâmetro é `onClick`, uma lambda comum, e `Text` é composable. A mensagem é `@Composable invocations can only happen from the context of a @Composable function`. Nomear o evento (`onClick = { ... }`) deixa claro qual é qual.

📖 Ref. Kotlin — Passing trailing lambdas: https://kotlinlang.org/docs/lambdas.html#passing-trailing-lambdas

3.3 Argumentos nomeados e valores padrão

As APIs do Compose têm muitos parâmetros opcionais. Argumentos nomeados e valores padrão tornam as chamadas legíveis:

```
OutlinedTextField(
  value = texto,
  onValueChange = onTextoChange,
  label = { Text("Nova tarefa") },
  isError = texto.isNotEmpty() && !valido,
  modifier = Modifier.weight(1f),
)
```

A mesma função tem dezenas de parâmetros (`enabled`, `singleLine`, `placeholder`...); todos têm valor padrão e só os necessários aparecem. A vírgula depois do último argumento é permitida e facilita reordenar linhas.

Uma convenção das APIs do Compose, que vale seguir nos seus componentes: o primeiro parâmetro opcional é `modifier: Modifier = Modifier`.

```
@Composable
fun CartaoTarefa(tarefa: Tarefa, onAlternar: () -> Unit, modifier: Modifier = Modifier) {
  Card(modifier.fillMaxWidth()) { /* ... */ }
}
```

Assim, quem usa o componente pode ajustar tamanho e espaçamento de fora, sem o componente prever cada caso.

Kotlin para quem conhece Java: substitui a pilha de sobrecargas e os builders. Em Java, uma API com 20 parâmetros opcionais precisaria de um builder; em Kotlin, basta dar padrão a cada um.

📖 Ref. Kotlin — Named arguments e default values: https://kotlinlang.org/docs/functions.html#named-arguments

3.4 Lambda com receptor: por que `weight` só funciona dentro de `Row`

No exemplo, `Modifier.weight(1f)` aparece dentro de uma `Row`. Fora dela, não compila:

```
Box { Text("x", Modifier.weight(1f)) }   // e: Unresolved reference 'weight'.
Row { Text("y", Modifier.weight(1f)) }   // compila
```

O motivo é um recurso do Kotlin: lambda com receptor. O conteúdo de `Row` tem tipo `RowScope.() -> Unit`, ou seja, dentro das chaves `this` é um `RowScope`. E `weight` é uma função de extensão de `Modifier` declarada dentro de `RowScope` (e outra dentro de `ColumnScope`). Só existe onde esse receptor existe.

```
Row { ... }        this: RowScope      → Modifier.weight, Modifier.align(Alignment.Vertical)
Column { ... }     this: ColumnScope   → Modifier.weight, Modifier.align(Alignment.Horizontal)
Box { ... }        this: BoxScope      → Modifier.align(Alignment), Modifier.matchParentSize
LazyColumn { ... } this: LazyListScope → item { }, items(...)
```

É o compilador impedindo um erro de layout: "peso" só faz sentido para distribuir espaço entre irmãos de uma linha ou coluna. O mesmo mecanismo explica por que `items(...)` só está disponível dentro de `LazyColumn`.

Kotlin para quem conhece Java: não há equivalente. O mais próximo seria passar um objeto de contexto como parâmetro da lambda (`row -> row.weight(...)`), e o Kotlin torna esse objeto implícito.

📖 Ref. Kotlin — Function literals with receiver: https://kotlinlang.org/docs/lambdas.html#function-literals-with-receiver

3.5 `val`, `var`, `data class` e `copy`

```
data class Tarefa(val id: Int, val titulo: String, val feita: Boolean = false)
```

- `val` não pode ser reatribuído (como `final`); `var` pode.
- `data` gera `equals`, `hashCode`, `toString` e `copy()` a partir das propriedades do construtor.
- `= false` é valor padrão: `Tarefa(1, "Estudar")` cria uma tarefa pendente.

Kotlin para quem conhece Java: é como um `record`, com dois extras: valores padrão e `copy()`.

`copy` cria um objeto novo alterando só o que foi pedido:

```
val t = Tarefa(1, "Estudar Compose")
val concluida = t.copy(feita = true)
println(t)           // Tarefa(id=1, titulo=Estudar Compose, feita=false)
println(concluida)   // Tarefa(id=1, titulo=Estudar Compose, feita=true)
println(t == Tarefa(1, "Estudar Compose"))   // true: igualdade por valor
```

A igualdade por valor (`==` chama `equals`) importa no Compose mais do que parece: é por ela que o estado decide se mudou (seção 5.1).

Um ponto que confunde: `val` protege a referência, não o conteúdo. `val lista = mutableListOf<Tarefa>()` não pode apontar para outra lista, mas pode ganhar itens com `add`.

> Erro comum: `var` nos campos da `data class` "para facilitar". `data class Tarefa(..., var feita: Boolean)` permite `tarefa.feita = true`, e a tela não muda (seção 5.1). No Compose, dados de UI são `val`; muda-se o estado, não o objeto.

📖 Ref. Kotlin — Data classes: https://kotlinlang.org/docs/data-classes.html

3.6 Null-safety e o smart cast que não acontece

Em Kotlin, a possibilidade de `null` faz parte do tipo: `Tarefa` nunca é nula; `Tarefa?` pode ser. Operadores para lidar com isso:

Operador	Exemplo	Resultado
`?.`	`selecionada?.titulo`	o título, ou `null`
`?:`	`selecionada?.titulo ?: "Nenhuma"`	o título, ou o texto padrão
`if (x != null)`	`if (t != null) t.titulo`	dentro do `if`, `t` é tratada como não nula (smart cast)
`!!`	`selecionada!!.titulo`	o título, ou `NullPointerException`

Kotlin para quem conhece Java: é a checagem que `Optional` e `@Nullable` tentam oferecer, feita pelo compilador em todo o código.

Numa tela com seleção, o estado costuma ser anulável, e aparece um erro que surpreende:

```
var selecionada by remember { mutableStateOf<Tarefa?>(null) }

if (selecionada != null) {
  Text(selecionada.titulo)
  // e: Smart cast to 'Tarefa' is impossible, because 'selecionada' is a delegated property.
}
```

O compilador não confia no `if`: `selecionada` é uma propriedade delegada (seção 3.7), e cada leitura chama `getValue()`, que poderia devolver outro valor. As saídas:

```
selecionada?.let { tarefa -> Text(tarefa.titulo) }   // 1. let com parâmetro não nulo

val atual = selecionada                              // 2. copiar para um val local
if (atual != null) Text(atual.titulo)
```

> Erro comum: resolver com `!!`. `Text(selecionada!!.titulo)` compila e funciona até o dia em que o estado for `null` no meio de uma recomposição. Prefira `?.let` ou o `val` local.

📖 Ref. Kotlin — Null safety: https://kotlinlang.org/docs/null-safety.html

3.7 Propriedades delegadas (`by`)

```
var contador by remember { mutableStateOf(0) }
```

`by` diz que ler e escrever a propriedade é delegado a outro objeto, aqui o `MutableState` devolvido por `remember`. O compilador traduz para algo como:

```
val contador$delegate = remember { mutableStateOf(0) }   // o MutableState<Int>
// ler contador       → contador$delegate.getValue(...)   → .value
// contador = 5       → contador$delegate.setValue(..., 5) → .value = 5
```

As três formas abaixo são equivalentes, segundo a documentação; escolha a que deixa o código mais legível:

```
val estado = remember { mutableStateOf(0) }          // estado.value, estado.value = 1
var valor by remember { mutableStateOf(0) }          // valor, valor = 1
val (atual, definir) = remember { mutableStateOf(0) } // atual, definir(1)
```

Kotlin para quem conhece Java: é um getter e um setter cujo corpo mora em outro objeto. Em Java, você escreveria `estado.getValue()` e `estado.setValue(1)` em cada uso.

Para o `by` funcionar com `MutableState`, as funções `getValue` e `setValue` precisam ser importadas:

```
import androidx.compose.runtime.getValue
import androidx.compose.runtime.setValue
```

> Erro comum: faltar o import do `getValue` / `setValue`. O erro de compilação não fala de import: `Property delegate must have a 'getValue(Nothing?, KMutableProperty0<*>)' method. None of the following functions is applicable`. Aceite a sugestão de import da IDE.

📖 Ref. Kotlin — Delegated properties: https://kotlinlang.org/docs/delegated-properties.html

3.8 Miudezas que aparecem no código

Kotlin	Java	Observação
`"Cliquei $contador"`, `"${tarefa.titulo}"`	`"Cliquei " + contador`	string template
`if (x) a else b` como valor	`x ? a : b`	`if` é expressão
`listOf(a, b)`	`List.of(a, b)`	lista somente leitura
`tarefas + nova`	copiar para um `ArrayList` e chamar `add`	nova lista com um item a mais
`tarefas.map { ... }`	`tarefas.stream().map(...).toList()`	direto na coleção
`tarefas.filterNot { it.id == id }`	`stream().filter(t -> t.id() != id)`	nova lista sem o item
`Tarefa(1, "X")`	`new Tarefa(1, "X")`	sem `new`
`12.dp`	—	propriedade de extensão sobre `Int`, cria um `Dp`
`::exitApplication`	`this::exitApplication`	referência de função

📖 Ref. Kotlin — Comparison to Java: https://kotlinlang.org/docs/comparison-to-java.html

---

4. Estado: `mutableStateOf`, `remember`, `rememberSaveable` e estado elevado

Estado é qualquer valor que pode mudar com o tempo e afeta o que a tela mostra: a lista de tarefas, o texto digitado, qual item está expandido. Este capítulo trata das três perguntas sobre cada estado: ele é observável? ele sobrevive à recomposição? onde ele deve morar?

4.1 `mutableStateOf`: um valor observável

```
interface MutableState<T> : State<T> {
  override var value: T
}
```

`mutableStateOf(valorInicial)` cria um `MutableState<T>`. O Compose registra quem lê `value` durante a composição; quando `value` recebe um valor novo, esses leitores são recompostos.

"Valor novo" tem um significado preciso: por padrão, `mutableStateOf` usa uma política de igualdade estrutural (`structuralEqualityPolicy()`). Atribuir um valor igual, por `equals`, ao atual não conta como mudança. A seção 5.1 mostra onde isso surpreende.

O campo de texto é o exemplo mais simples de por que o estado precisa ser observável:

```
var texto by remember { mutableStateOf("") }

OutlinedTextField(
  value = texto,                    // leitura: o campo passa a depender de 'texto'
  onValueChange = { texto = it },   // escrita: agenda recomposição de quem lê 'texto'
)
```

Digitar chama `onValueChange`, que muda `texto`, que recompõe o campo com o novo `value`. Sem o estado, o campo ficaria vazio (seção 1.7).

📖 Ref. Android Developers — State and Jetpack Compose: https://developer.android.com/develop/ui/compose/state

4.2 `remember`: sobreviver à recomposição

Uma composable é uma função: variáveis locais são recriadas a cada execução. `remember` guarda um valor na composição durante a composição inicial e devolve o mesmo valor nas recomposições seguintes. O valor é esquecido quando a composable sai da composição (por exemplo, quando um `if` deixa de mostrá-la).

As duas peças têm papéis diferentes: `mutableStateOf` faz a tela reagir; `remember` faz o valor persistir. O teste abaixo mostra as combinações, cada uma num botão que incrementa um número ao ser clicado duas vezes:

Código	Depois de 2 cliques	Depois de o pai recompor a função
`var n by remember { mutableStateOf(0) }`	2	2
`var n by mutableStateOf(0)`, lido no corpo da função	0	0
`var n by mutableStateOf(0)`, lido só dentro do `Button { }`	2	volta a 0
`val c = remember { Caixa() }; c.n++` (objeto comum)	0	2
`var n = 0` (variável local)	0	0

Leitura linha a linha:

- A primeira é o padrão: observável e lembrado.

- Sem `remember`, o estado é recriado com 0 a cada execução da função. Se o estado é lido no corpo da função, o clique recompõe a função, e o valor volta a 0 na hora.
- A terceira linha é a armadilha: lido só dentro do conteúdo do botão, o clique recompõe apenas esse conteúdo (seção 1.3), que ainda enxerga o estado antigo. Parece funcionar. Quando a função inteira recompõe por outro motivo (o pai passou um parâmetro novo), o estado é recriado e o valor some.
- Com `remember` e um objeto comum, o valor é guardado, mas ninguém avisa o Compose. A tela fica parada e, quando algo recompõe a função por outro motivo, o valor aparece de repente.
- A variável local não guarda nem avisa.

O lint do Android acusa a segunda e a terceira formas como erro, `UnrememberedMutableState: Creating a state object during composition without using remember`. No exemplo, o `lintDebug` apontou o erro num arquivo de `androidMain`, mas não o mesmo código colocado em `commonMain`; em projeto KMP, não conte com o lint para pegar esse caso.

O mesmo lint sugere, para números, `mutableIntStateOf(0)` no lugar de `mutableStateOf(0)`, que evita converter o `Int` em objeto a cada escrita. O `proximoId` do exemplo é um candidato.

> Erro comum: estado sem `remember` que "funciona às vezes". A terceira linha da tabela passa no teste manual e falha quando a tela ganha um parâmetro que muda. Todo `mutableStateOf` dentro de uma composable vai dentro de `remember` (ou `rememberSaveable`).

📖 Ref. Android Developers — State in composables:
https://developer.android.com/develop/ui/compose/state#state-in-composables

4.3 `remember` com chave

`remember(chave) { ... }` recalcula o valor quando a chave muda. Serve para dois casos:

```
// 1. Cálculo caro que depende de parâmetros
val visiveis = remember(tarefas, filtro) { tarefas.filter { filtro.aceita(it) } }

// 2. Estado inicializado a partir de um parâmetro
@Composable
fun EditorDeTitulo(tarefa: Tarefa) {
  var titulo by remember(tarefa) { mutableStateOf(tarefa.titulo) }
  // ...
}
```

O segundo caso tem um erro clássico. Sem a chave, `remember { mutableStateOf(tarefa.titulo) }` roda uma vez só; se o pai passar outra tarefa, o editor continua mostrando o título antigo. No teste, depois de o pai trocar o título de "Antigo" para "Novo", a versão sem chave mostrou "Antigo" e a versão com `remember(tarefa)` mostrou "Novo".

Para cálculos baratos, como `texto.isNotBlank()`, não é preciso `remember`: calcule direto no corpo da função (seção 4.7).

4.4 `rememberSaveable`: sobreviver à recriação da Activity

`remember` guarda o valor na composição. Se a Activity for destruída e recriada, a composição começa do zero e o valor se perde. Isso acontece em dois casos no Android:

- mudanças de configuração que a Activity não trata (por padrão, rotação, tema, idioma...);
- morte do processo pelo sistema, quando o app está em segundo plano e o aparelho precisa de memória.

`rememberSaveable` guarda o valor no mecanismo de estado salvo da Activity (um `Bundle`) e o restaura na recriação. Conferido com `StateRestorationTester` no alvo Android (Robolectric):

```
var a by remember { mutableStateOf(0) }
var b by rememberSaveable { mutableStateOf(0) }
// dois cliques, depois recriação simulada:
// remember=0 saveable=2
```

O que pode ser salvo: tipos que cabem num `Bundle` (primitivos, `String` e afins). Uma lista de `Tarefa` não cabe; a tentativa falhou com `IllegalArgumentException: Parcel: unknown type for value Tarefa(id=1, titulo=A, feita=false)`. Para tipos próprios, há três caminhos:

Caminho	Onde funciona	Exemplo
`@Parcelize` na classe	só Android	não serve em `commonMain`
`listSaver` / `mapSaver`	multiplataforma	abaixo
salvar só o id e reconstruir o resto	multiplataforma	recomendado para objetos grandes

```
val TarefaSaver = listSaver<Tarefa, Any>(
  save = { listOf(it.id, it.titulo, it.feita) },
  restore = { Tarefa(it[0] as Int, it[1] as String, it[2] as Boolean) },
)

var tarefa by rememberSaveable(stateSaver = TarefaSaver) { mutableStateOf(Tarefa(1, "A")) }
```

Conferido: com o `listSaver`, a tarefa alterada foi restaurada depois da recriação. (Os testes com Robolectric imprimiram esses resultados; a execução terminou com um erro de ambiente, `Unsupported class file major version 70`, que vem de rodar o Robolectric com Java 26 no computador de teste e acontece depois das verificações.)

A documentação recomenda guardar o mínimo no `Bundle` (ids, texto digitado, posição de rolagem), porque o tamanho é limitado e objetos grandes causam `TransactionTooLargeException`.

Um detalhe do exemplo: o `AndroidManifest.xml` declara `android:configChanges="orientation|screenSize|screenLayout|keyboardHidden|density|uiMode"`. Com isso, rotação, mudança de tamanho de tela e troca de tema claro/escuro não recriam a Activity; ela recebe `onConfigurationChanged()` e o Compose recompõe. Nesses casos, até o `remember` sobrevive. Outras mudanças (idioma, por exemplo) e a morte do processo continuam recriando, e aí só o `rememberSaveable` preserva o valor.

No alvo Desktop não existe Activity nem recriação por mudança de configuração; na prática, `rememberSaveable` se comporta como `remember` durante a execução.

> Erro comum: `rememberSaveable` com a lista inteira de dados. Além de estourar o `Bundle` com listas grandes, tipos próprios quebram em execução no Android, não na compilação. Salve texto digitado e seleção; dados de domínio vêm de outra fonte (na próxima etapa do projeto, um repositório ou a API).

📖 Ref. Android Developers — Save UI state in Compose:
https://developer.android.com/develop/ui/compose/state-saving

📖 Ref. Android Developers — `<activity>`, atributo `android:configChanges`:
https://developer.android.com/guide/topics/manifest/activity-element

4.5 Stateful × stateless e o estado elevado

Uma composable que chama `remember` para guardar estado é stateful: controla o próprio estado. Uma que só recebe dados e emite eventos é stateless.

Antes, com o texto preso dentro do formulário:

```
@Composable
fun FormularioTarefa(onAdicionar: (String) -> Unit) {
  var texto by remember { mutableStateOf("") }     // estado preso aqui
  Row {
    OutlinedTextField(value = texto, onValueChange = { texto = it })
    Button(onClick = { onAdicionar(texto); texto = "" }, enabled = texto.isNotBlank()) {
      Text("Adicionar")
    }
  }
}
```

Funciona, mas: a tela não consegue limpar ou preencher o campo por fora; um preview só mostra o campo vazio; e testar "botão desabilitado com texto em branco" exige digitar.

Depois, como está no exemplo:

```
@Composable
fun FormularioTarefa(
  texto: String,
  onTextoChange: (String) -> Unit,
  valido: Boolean,
  onAdicionar: () -> Unit,
) {
  Row(verticalAlignment = Alignment.CenterVertically) {
    OutlinedTextField(
      value = texto,
      onValueChange = onTextoChange,
      label = { Text("Nova tarefa") },
      isError = texto.isNotEmpty() && !valido,
      modifier = Modifier.weight(1f),
    )
    Spacer(Modifier.width(8.dp))
    Button(onClick = onAdicionar, enabled = valido) { Text("Adicionar") }
  }
}
```

Elevar o estado é trocar a variável de estado por dois parâmetros: o valor atual (`texto: String`) e um evento que pede a mudança (`onTextoChange: (String) -> Unit`). O estado sobe para quem chama, a `App()`.

O que se ganha, segundo a documentação:

Propriedade	No exemplo
fonte única da verdade	só a `App()` guarda `texto`; não há cópia no formulário
encapsulamento	só a `App()` altera `texto`
compartilhável	a `App()` usa `texto` também para criar a tarefa
interceptável	a `App()` poderia ignorar ou transformar o texto (limitar tamanho) antes de gravar
desacoplado	o formulário não sabe onde o texto mora; pode ir para um `ViewModel` depois

`CartaoTarefa` segue o mesmo desenho: recebe `tarefa` e devolve `onAlternar`. Como não guarda nada, o preview `CartaoTarefaPreview` o mostra marcado só passando `feita = true`.

📖 Ref. Android Developers — State hoisting: https://developer.android.com/develop/ui/compose/state-hoisting

4.6 Onde o estado deve morar

A documentação dá três regras para decidir até onde elevar:

1. No mínimo até o ancestral comum mais baixo de todas as composables que leem o estado.
2. No mínimo até o nível mais alto em que ele é alterado.

3. Dois estados que mudam em resposta aos mesmos eventos sobem juntos.

Aplicando ao exemplo:

- `tarefas` é lida pela lista e alterada pelo formulário (adicionar) e pelos cartões (alternar). O ancestral comum é `App()`.
- `texto` é lido e alterado pelo formulário, mas a `App()` também o lê (para criar a tarefa) e o altera (para limpar). Sobe para `App()`.
- `proximoId` muda no mesmo evento que `tarefas` (adicionar): sobe junto.

Elevar mais do que o necessário é possível; elevar de menos é o que quebra o fluxo de dados.

4.7 Fluxo de dados unidirecional

Com o estado na `App()`, o ciclo completo de "adicionar tarefa":

```
1. App() compõe:  texto = "",  tarefas = [A, B]
       │ estado desce
       ▼
2. FormularioTarefa mostra o campo vazio, botão desabilitado (valido = false)
3. usuário digita "Ler"  ──► onTextoChange("Ler") sobe
4. App():  texto = "Ler"  ──► recomposição
5. FormularioTarefa mostra "Ler", botão habilitado (valido = true)
6. usuário clica  ──► onAdicionar() sobe
7. App():  tarefas = tarefas + Tarefa(3, "Ler");  proximoId++;  texto = ""
8. recomposição: lista com 3 cartões, campo vazio
```

Conferido com um teste de UI na `App()` do exemplo: o botão fica desabilitado com o campo vazio e com espaços em branco, habilita com texto, a tarefa criada aparece na lista e o campo volta vazio.

Note `val tituloValido = texto.isNotBlank()` na `App()`. É estado derivado: um valor calculado a partir de outro estado, recalculado a cada recomposição. Guardá-lo num `mutableStateOf` separado criaria uma segunda fonte da verdade, que poderia discordar do texto.

📖 Ref. Android Developers — Architecting your Compose UI (fluxo unidirecional): https://developer.android.com/develop/ui/compose/architecture

4.8 Erros comuns com estado

> Erro comum: copiar um parâmetro para um estado local sem chave. `var titulo by remember { mutableStateOf(tarefa.titulo) }` ignora mudanças posteriores em `tarefa` (seção 4.3). Use `remember(tarefa)` ou, melhor, eleve o estado.

> Erro comum: guardar estado derivado. `var valido by remember { mutableStateOf(false) }` atualizado em `onValueChange` é uma segunda fonte da verdade. Calcule `val valido = texto.isNotBlank()` no corpo da função.

> Erro comum: estado elevado de menos. Se dois componentes irmãos precisam do mesmo valor e cada um guarda o seu, eles vão divergir. Suba para o pai comum.

> Erro comum: o componente recebe o `MutableState` inteiro. `fun Formulario(texto: MutableState<String>)` funciona, mas deixa o filho alterar o estado sem passar pelo pai, e acopla o componente ao tipo do Compose. Passe valor e evento.

> Erro comum: `rememberSaveable` com tipo que não cabe no `Bundle`. Compila, roda no Desktop e quebra no Android ao salvar o estado (seção 4.4).

---

5. Imutabilidade: `val`, `copy` e reatribuição

O capítulo 4 disse que a tela reage quando um estado observável recebe um valor novo. Este capítulo tira a consequência: se o que muda é o conteúdo de um objeto, e não o valor do estado, o Compose não fica sabendo. Por isso o exemplo usa dados imutáveis e reatribui o estado.

5.1 O que o Compose vê e o que não vê

O Compose só percebe escritas em objetos de estado do próprio Compose (`MutableState`, `SnapshotStateList`, `SnapshotStateMap`). Mudar um campo dentro de um objeto comum não é uma escrita observável.

Experimentos com uma tela que mostra "feita=..." da primeira tarefa. Todos foram rodados em teste de UI no Desktop:

Ação	A tela atualizou?	Por quê
`tarefas[0].feita = true` (campo `var`, lista em `mutableStateOf`)	não	nenhuma escrita em estado
em seguida, `tarefas = tarefas`	não	mesmo objeto: o valor não mudou

Ação	A tela atualizou?	Por quê
em seguida, `tarefas = tarefas.toList()`	não	lista nova, mas igual por `equals` à anterior (os itens são os mesmos objetos); a política de igualdade estrutural descarta a escrita
`lista.value.add("y")` em `mutableStateOf(mutableListOf(...))`	não	a lista interna mudou; o estado continua apontando para ela
`tarefas = tarefas.map { if (...) it.copy(feita = !it.feita) else it }`	sim	lista nova, diferente por `equals`
`add` em `mutableStateListOf`	sim	`SnapshotStateList` é observável
`lista[i] = lista[i].copy(feita = true)` em `mutableStateListOf`	sim	substituir um item é escrita observável
`lista[0].feita = true` em `mutableStateListOf` (campo `var` do item)	não	a lista não mudou; o item mudou por dentro

Um detalhe perigoso da última linha: depois dela, quando um `add` recompôs a tela por outro motivo, o texto passou a mostrar `feita=true`. A mudança "aparece por acaso" na próxima recomposição. É o tipo de defeito que ninguém consegue reproduzir de propósito.

A documentação do Compose alerta exatamente para isso: objetos mutáveis não observáveis, como `ArrayList` ou uma data class com `var`, fazem o usuário ver dados errados ou desatualizados.

📖 Ref. Android Developers — State and Jetpack Compose (aviso sobre objetos mutáveis): https://developer.android.com/develop/ui/compose/state

5.2 O padrão do exemplo: `val` + `copy` + reatribuição

```
data class Tarefa(val id: Int, val titulo: String, val feita: Boolean = false)

var tarefas by remember { mutableStateOf(listOf(Tarefa(1, "Estudar Compose"))) }
```

Dois níveis, com papéis diferentes:

- `var tarefas` é o estado. Ele muda, por reatribuição.
- `List<Tarefa>` e cada `Tarefa` são imutáveis. Nunca mudam; são substituídos.

As operações de uma lista de tarefas nesse estilo:

```
// adicionar
tarefas = tarefas + Tarefa(proximoId, texto.trim())

// alternar 'feita' de uma tarefa
tarefas = tarefas.map { if (it.id == alvo) it.copy(feita = !it.feita) else it }

// renomear
tarefas = tarefas.map { if (it.id == alvo) it.copy(titulo = novoTitulo) else it }

// remover
tarefas = tarefas.filterNot { it.id == alvo }
```

Cada linha cria uma lista nova e a atribui ao estado. A atribuição é a escrita que o Compose observa; a lista nova é diferente da anterior por `equals` (um item tem outro valor, ou há um item a mais ou a menos), então a recomposição acontece.

Kotlin para quem conhece Java: é o mesmo raciocínio de trabalhar com `List.of(...)` e `record`: em vez de `setFeita(true)`, cria-se outro objeto.

5.3 O `map` do exemplo, linha a linha

```
onAlternar = {
  tarefas = tarefas.map {
    if (it.id == tarefa.id) it.copy(feita = !it.feita) else it
  }
}
```

Com `tarefas = [A(1, feita=false), B(2, feita=false), C(3, feita=false)]` e o clique no cartão B (`tarefa.id == 2`):

Volta do `map`	`it`	`it.id == 2` ?	Devolve	Mesmo objeto de antes?
1	A	não	`it` (A)	sim
2	B	sim	`B.copy(feita = true)`	não, objeto novo
3	C	não	`it` (C)	sim

Resultado: uma lista nova `[A, B', C]`, atribuída a `tarefas`.

Em uma frase: "para cada tarefa, se for a clicada, devolvo uma cópia com `feita` invertido; senão, devolvo ela mesma; a lista resultante vira o novo estado".

Os dois nomes do trecho: `tarefa` é o parâmetro do `items` (a linha em que se clicou, capturada pela lambda `onAlternar`); `it` é cada item percorrido pelo `map`.

5.4 Por que isso deixa a lista rápida

A tabela acima mostra que A e C continuam sendo os mesmos objetos. Isso tem efeito direto na recomposição. Quando a lista nova chega:

- o cartão B recebe um `Tarefa` diferente (por `equals`) e recompõe;
- os cartões A e C recebem objetos iguais aos anteriores e são pulados.

Foi o resultado medido na seção 1.3: depois de alternar B, só `cartao-2` executou.

O Compose consegue pular com segurança porque `Tarefa` é estável: uma classe com propriedades `val` de tipos imutáveis não muda depois de criada, então "igual antes, igual agora" basta. Segundo a documentação de estabilidade, uma classe com `var` é considerada instável, porque o Compose não teria como saber se ela mudou por dentro.

Custo: cada operação copia a lista (percorre n itens) e cria um objeto por item alterado. Para listas de tela (dezenas ou centenas de itens), é irrelevante. Para coleções muito grandes com alterações frequentes, existem coleções persistentes (`kotlinx.collections.immutable`), que a documentação do Compose cita como opção.

📖 Ref. Android Developers — Stability in Compose:
https://developer.android.com/develop/ui/compose/performance/stability

5.5 `mutableStateListOf`: a alternativa observável

`mutableStateListOf` cria uma `SnapshotStateList`, uma lista mutável cujas operações são observadas:

```
val tarefas = remember { mutableStateListOf(Tarefa(1, "Estudar Compose")) }

tarefas.add(Tarefa(2, "Entender estado"))                    // observado
val i = tarefas.indexOfFirst { it.id == alvo }
tarefas[i] = tarefas[i].copy(feita = !tarefas[i].feita)      // observado
tarefas.removeAll { it.id == alvo }                          // observado
```

Repare no `val`: a referência para a lista não muda; o conteúdo muda por operações observáveis. E os itens continuam imutáveis: substitui-se o item com `copy`, não se altera um campo.

	`mutableStateOf(listOf(...))` + reatribuição	**`mutableStateListOf(...)`**
Declaração	`var tarefas by remember { ... }`	`val tarefas = remember { ... }`
Adicionar	`tarefas = tarefas + nova`	`tarefas.add(nova)`
Alterar item	`tarefas = tarefas.map { ... }`	`tarefas[i] = tarefas[i].copy(...)`
Custo por operação	copia a lista	altera no lugar

	`mutableStateOf(listOf(...))` + reatribuição	`mutableStateListOf(...)`
Risco	esquecer de reatribuir	passar a lista mutável adiante e alguém alterar sem querer

O exemplo usa a primeira forma: cada mudança é uma atribuição visível, e a lista passada aos componentes é uma `List` somente leitura. Com `mutableStateListOf`, passe aos filhos `List<Tarefa>` e eventos, não a `SnapshotStateList`, para manter o fluxo unidirecional.

📖 Ref. Android Developers — `SnapshotStateList`:
https://developer.android.com/reference/kotlin/androidx/compose/runtime/snapshots/SnapshotStateList

5.6 Erros comuns de imutabilidade

> Erro comum: `var` nos campos da data class. `tarefa.feita = true` não recompõe nada (seção 5.1), e a mudança pode aparecer depois, por acaso. Campos de dados de UI são `val`.

> Erro comum: `mutableStateOf(mutableListOf())` e `add`. O estado guarda a referência da lista; `add` altera a lista sem escrever no estado. Use `mutableStateOf(listOf())` com reatribuição, ou `mutableStateListOf()`.

> Erro comum: mutar e depois reatribuir "para forçar". Depois de alterar um objeto por dentro, `tarefas = tarefas.toList()` também não recompôs no teste: a lista nova é igual à antiga por `equals`. Não há atalho; crie o valor novo com `copy`.

> Erro comum: gerar id dentro da composição. `Tarefa(proximoId++, ...)` no corpo da composable é efeito colateral (seção 1.4): pode rodar várias vezes. Crie a tarefa no callback do evento, como o `onAdicionar` do exemplo.

> Erro comum: comparar objetos por referência. Em Kotlin, `==` chama `equals` (valor) e `===` compara referência. Duas tarefas com os mesmos campos são `==` mesmo sendo objetos diferentes; é a mesma regra que o estado usa para decidir se mudou.

---

6. Componentes e `Modifier`

A rubrica da Sprint 1 pede telas com composição limpa e componentes próprios reutilizáveis. Este capítulo trata das duas ferramentas para isso: funções `@Composable` bem desenhadas e `Modifier`, que ajusta tamanho, espaçamento e comportamento sem que o componente precise prever cada caso.

6.1 O que faz um bom componente

O exemplo tem dois componentes próprios, `CartaoTarefa` e `FormularioTarefa`; o MUSI tem `CartaoObra` e `FiltrosRapidos`. Eles seguem as mesmas regras:

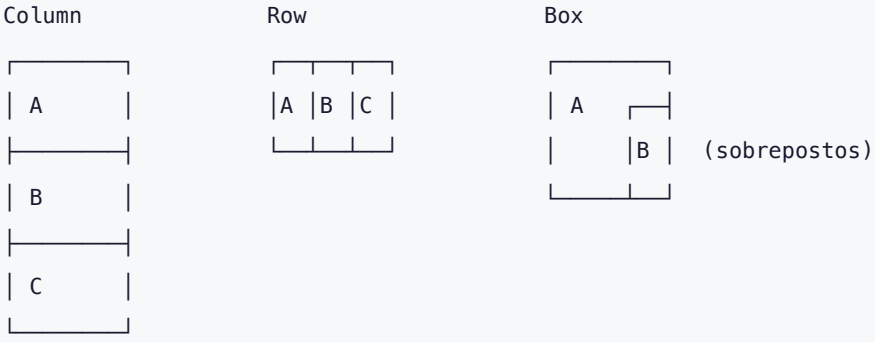
Regra	Em `CartaoTarefa`	Por quê
recebe dados por parâmetro	`tarefa: Tarefa`	pode ser usado com qualquer tarefa, inclusive num preview
devolve eventos por lambda	`onAlternar: () -> Unit`	quem usa decide o que acontece
não guarda estado de dados	sem `remember` para a tarefa	uma só fonte da verdade (capítulo 4)
aceita `modifier: Modifier = Modifier`	no MUSI, `CartaoObra(obra, modifier)`	quem usa ajusta tamanho e espaçamento de fora
aplica o `modifier` no elemento raiz	`Card(modifier.fillMaxWidth())`	o ajuste externo vale para o componente inteiro
nome pelo que é, não por onde aparece	`CartaoTarefa`, não `ItemDaListaDaTelaPrincipal`	serve em outras telas

Estado de interface puramente visual e local (um cartão expandido ou não, por exemplo) pode ficar dentro do componente, desde que ninguém de fora precise controlá-lo. Se a tela precisar "recolher todos", o estado sobe.

📖 Ref. Android Developers — Compose layout basics:
https://developer.android.com/develop/ui/compose/layouts/basics

6.2 `Column`, `Row` e `Box`

Três layouts cobrem a maior parte das telas:



Cada um posiciona os filhos em dois eixos, com parâmetros diferentes:

Layout	Eixo principal (`Arrangement`)	Eixo cruzado (`Alignment`)
`Column`	vertical: `verticalArrangement`	horizontal: `horizontalAlignment`
`Row`	horizontal: `horizontalArrangement`	vertical: `verticalAlignment`
`Box`	—	`contentAlignment`

```
Row(
  horizontalArrangement = Arrangement.spacedBy(8.dp),       // 8dp entre os filhos
  verticalAlignment = Alignment.CenterVertically,           // centralizados na altura
) {
  Checkbox(checked = tarefa.feita, onCheckedChange = { onAlternar() })
  Text(tarefa.titulo, Modifier.weight(1f))                  // ocupa o espaço que sobra
  TextButton(onClick = onEditar) { Text("Editar") }
}
```

`weight` distribui o espaço restante na proporção dos pesos. Conferido: numa `Row` de 300dp, dois filhos com `weight(1f)` e `weight(2f)` ficaram com 100dp e 200dp. `Spacer` é um elemento vazio usado só para ocupar espaço (`Spacer(Modifier.width(8.dp))`), como no exemplo.

6.3 `Modifier` : uma cadeia de decorações

`Modifier` é um valor imutável que acumula instruções, na ordem em que são encadeadas:

```
Modifier
  .fillMaxWidth()          // largura: toda a disponível
  .padding(12.dp)          // espaço interno em volta do conteúdo
  .clickable { abrir() }   // área que reage a toque
```

Modificadores frequentes:

Grupo	Exemplos
tamanho	`size(48.dp)` , `width` , `height` , `fillMaxWidth()` , `fillMaxSize()` , `weight(1f)` (só em `Row` / `Column`)
espaço	`padding(16.dp)` , `padding(horizontal = 16.dp, vertical = 8.dp)`
aparência	`background(cor)` , `clip(RoundedCornerShape(8.dp))` , `border(1.dp, cor)`
comportamento	`clickable { }` , `verticalScroll(estado)` , `horizontalScroll(estado)`
testes e acessibilidade	`testTag("cartao")` , `semantics { }`

Kotlin para quem conhece Java: a cadeia lembra um builder (`new Estilo().largura(...).margem(...)`), com uma diferença importante: não existe um objeto final "resolvido"; a ordem das chamadas é a própria semântica.

📖 Ref. Android Developers — Compose modifiers: https://developer.android.com/develop/ui/compose/modifiers

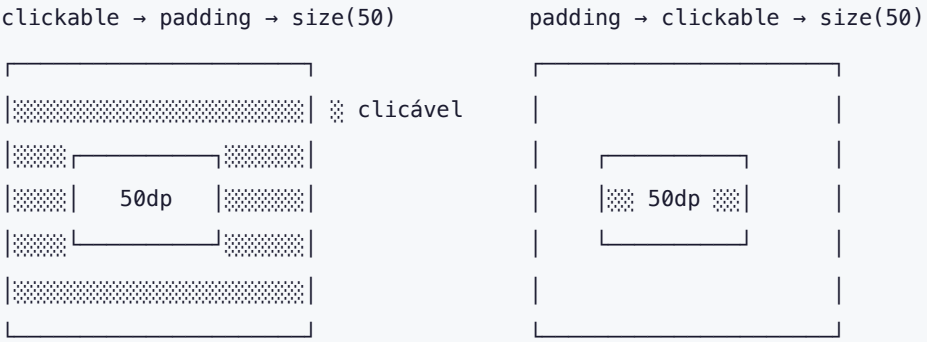
6.4 A ordem importa

Cada modificador age sobre o resultado dos anteriores. Não há "margem" e "padding" separados, como no modelo de caixas das Views ou do CSS: há só `padding`, e sua posição na cadeia decide o efeito. Dois casos conferidos em teste:

```
Box(Modifier.size(100.dp).padding(10.dp))   // ocupa 100 x 100; o conteúdo fica em 80 x 80
Box(Modifier.padding(10.dp).size(100.dp))   // ocupa 120 x 120; o conteúdo fica em 100 x 100
```

```
Box(Modifier.clickable { a++ }.padding(16.dp).size(50.dp))   // a borda de 16dp responde ao toque
Box(Modifier.padding(16.dp).clickable { b++ }.size(50.dp))   // só o quadrado de 50dp responde
```

Um toque a 5px da borda externa incrementou `a` e não incrementou `b`. A regra de leitura: tudo que vem antes de `clickable` fica dentro da área clicável.



O mesmo raciocínio vale para `background`: `background(azul).padding(8.dp)` pinta a área inteira e afasta o conteúdo; `padding(8.dp).background(azul)` afasta primeiro e pinta só a parte de dentro.

📖 Ref. Android Developers — Order of modifiers matters:
https://developer.android.com/develop/ui/compose/modifiers#order-modifier-matters

6.5 Modificadores com escopo

Alguns modificadores só existem dentro de um layout específico (seção 3.4): `weight` dentro de `Row` e `Column`; `align` com parâmetros diferentes em `Row`, `Column` e `Box`; `matchParentSize` em `Box`. Fora do escopo, o código não compila (`Unresolved reference 'weight'`). É o compilador garantindo que o modificador faz sentido onde foi usado.

Quando um componente próprio precisa desses modificadores no conteúdo que recebe, o slot pode declarar o escopo (próxima seção).

6.6 Slots: componentes que recebem UI

Um slot é um parâmetro do tipo `@Composable () -> Unit`: o componente reserva um lugar e quem usa decide o que vai nele. `Button { }`, `Card { }` e o `label` do `OutlinedTextField` são slots.

Um cartão com área de ações configurável:

```
@Composable
fun CartaoComAcoes(
  titulo: String,
  modifier: Modifier = Modifier,
  acoes: @Composable RowScope.() -> Unit = {},
) {
  Card(modifier.fillMaxWidth()) {
    Row(Modifier.padding(12.dp), verticalAlignment = Alignment.CenterVertically) {
      Text(titulo, Modifier.weight(1f))
      acoes()                                  // o que quem usa passou
    }
  }
}

// uso
CartaoComAcoes("Estudar Compose") {
  TextButton(onClick = onEditar) { Text("Editar") }
  IconButton(onClick = onRemover) { Text("×") }
}
```

`RowScope.()` no tipo do slot faz o conteúdo rodar dentro da `Row`, então quem usa pode aplicar `Modifier.weight` nas ações. O valor padrão `{}` torna o slot opcional.

O layout de tela do Material 3, `Scaffold`, é todo feito de slots:

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun TelaDeTarefas() {
  Scaffold(
    topBar = { TopAppBar(title = { Text("Tarefas") }) },
    floatingActionButton = { FloatingActionButton(onClick = { /* ... */ }) { Text("+") } },
  ) { espacamento ->
    LazyColumn(contentPadding = espacamento) { /* ... */ }
  }
}
```

Dois detalhes do código:

- `TopAppBar` ainda é API experimental do Material 3. Sem `@OptIn(ExperimentalMaterial3Api::class)`, a compilação falha com `This material API is experimental and is likely to change or to be removed in the future.` (conferido).
- O `Scaffold` passa `PaddingValues` para o conteúdo. A documentação pede que esse espaçamento seja aplicado no elemento raiz do conteúdo; sem ele, a lista fica escondida atrás da barra superior.

📖 Ref. Android Developers — Scaffold: https://developer.android.com/develop/ui/compose/components/scaffold

6.7 Erros comuns com componentes e modifiers

> Erro comum: receber `modifier` e não usar, ou usar no filho errado. `fun Cartao(modifier: Modifier = Modifier) { Card(Modifier.fillMaxWidth()) { Text(..., modifier) } }` aplica o ajuste externo só ao texto. O `modifier` recebido vai no elemento raiz.

> Erro comum: esperar que o padding antes do `clickable` seja clicável. Conferido na seção 6.4: só o que vem antes de `clickable` na cadeia faz parte da área de toque.

> Erro comum: tamanhos fixos em tudo. `width(360.dp)` funciona no emulador escolhido e quebra em outra largura. Prefira `fillMaxWidth()`, `weight` e deixe o conteúdo definir a altura. Layouts para várias larguras são assunto de 21/09.

> Erro comum: ignorar o `PaddingValues` do `Scaffold`. O conteúdo passa por baixo da `TopAppBar`. Aplique o valor recebido no conteúdo.

> Erro comum: um componente para cada tela. `CartaoTarefaDaTelaInicial` e `CartaoTarefaDaBusca` quase iguais. Um componente com parâmetros (e slots, se preciso) atende os dois e evita a "repetição de código" que a rubrica penaliza.

---

7. Listas: `LazyColumn`, `items` e `key`

7.1 Por que "lazy"

Uma lista pode ser feita com `Column` e um laço:

```
Column(Modifier.verticalScroll(rememberScrollState())) {
  for (tarefa in tarefas) CartaoTarefa(tarefa, onAlternar = { /* ... */ })
}
```

Funciona para poucos itens, mas compõe e mede todos, visíveis ou não. `LazyColumn` compõe só os que cabem na tela (e cria os próximos conforme a rolagem). Conferido com 1.000 itens de 50dp numa área de 400dp de altura:

Implementação	Itens compostos
`Column` + `verticalScroll`	1.000
`LazyColumn`	8

A família lazy:

Componente	Uso
`LazyColumn`	lista vertical
`LazyRow`	lista horizontal (carrossel)
`LazyVerticalGrid`, `LazyHorizontalGrid`	grade com colunas (ou linhas)
`LazyVerticalStaggeredGrid`	grade com itens de alturas diferentes

📖 Ref. Android Developers — Lists and grids: https://developer.android.com/develop/ui/compose/lists

7.2 A DSL de conteúdo: `item`, `items`, `itemsIndexed`

O conteúdo de uma `LazyColumn` não é uma lambda `@Composable` comum, e sim `LazyListScope.() -> Unit`: uma DSL que descreve os itens. A lista decide quando compor cada um.

```
LazyColumn {
  item { Text("Pendentes", style = MaterialTheme.typography.titleMedium) }   // um item

  items(pendentes, key = { it.id }) { tarefa ->                              // um item por elemento
    CartaoTarefa(tarefa, onAlternar = { alternar(tarefa.id) })
  }

  item { Text("Concluídas", style = MaterialTheme.typography.titleMedium) }

  itemsIndexed(concluidas, key = { _, t -> t.id }) { indice, tarefa ->       // com a posição
    Text("${indice + 1}. ${tarefa.titulo}")
  }
}
```

Cabeçalho, listas e rodapé ficam na mesma `LazyColumn`. É a forma recomendada de combinar conteúdo, em vez de colocar uma lista dentro de uma coluna rolável (seção 7.8).

Kotlin para quem conhece Java: `items(...)` só existe dentro das chaves de `LazyColumn`, pelo mesmo mecanismo de lambda com receptor que restringe `weight` a `Row` (seção 3.4).

7.3 `key`: identidade estável para cada item

Sem `key`, a lista identifica cada item pela posição. Quando itens entram, saem ou mudam de ordem, o estado lembrado de uma posição passa a pertencer a outro item. Conferido com itens que guardam `expandido` em `remember`:

```
lista [1, 2, 3]; expandir o item 2; remover o item 1

sem key:  item 2 expandido=false, item 3 expandido=true    ← o estado ficou na posição
com key:  item 2 expandido=true,  item 3 expandido=false   ← o estado acompanhou o item
```

Com `key = { it.id }`, o Compose associa o estado ao `id` e o move junto com o item. Isso vale também para animações de itens e para preservar a posição de rolagem quando a lista muda.

Três regras para a chave:

- Única. Duas tarefas com o mesmo `id` derrubam a tela com `IllegalArgumentException: Key "1" was already used. If you are using LazyColumn/Row please make sure you provide a unique key for each item.` (conferido).
- Estável. O `id` da tarefa, não o índice nem o título (que pode ser editado).
- Salvável no Android. A documentação exige um tipo suportado por `Bundle` (primitivos, `String`, enums, `Parcelable`), porque a chave é usada para restaurar o `rememberSaveable` dos itens.

📖 Ref. Android Developers — Item keys: https://developer.android.com/develop/ui/compose/lists#item-keys

7.4 Estado dentro de um item

Um item que sai da tela sai da composição. O que ele guardava em `remember` é descartado; o que guardava em `rememberSaveable` é salvo e restaurado quando ele volta. Conferido com valores aleatórios gerados em cada forma, rolando até o item 90 e voltando:

```
antes:          item 1 remember=550 saveable=256
item 1 saiu da composição (descartado)
depois voltar:  item 1 remember=224 saveable=256
```

Para estado de interface que o usuário espera manter (um cartão expandido), use `rememberSaveable` no item, com `key` na lista. Se a informação importa para a tela (quantos cartões estão expandidos, "recolher todos"), eleve-a: guarde, por exemplo, um conjunto de ids expandidos no estado da tela.

7.5 Espaçamento: `contentPadding` e `Arrangement`

```
LazyColumn(
  contentPadding = PaddingValues(16.dp),              // espaço nas bordas do conteúdo
  verticalArrangement = Arrangement.spacedBy(8.dp),   // espaço entre itens
) { /* ... */ }
```

Conferido: o primeiro item começou a 16dp do topo e da esquerda; o segundo, logo abaixo, somando a altura do primeiro e os 8dp de espaço.

Por que `contentPadding` e não `Modifier.padding`? `Modifier.padding(16.dp)` encolhe a área da lista: os itens são cortados 16dp antes da borda ao rolar. `contentPadding` mantém a área de rolagem inteira e só afasta o primeiro e o último item das bordas. É também como o espaçamento do `Scaffold` deve ser aplicado a uma lista (seção 6.6).

Para listas com tipos de item diferentes (cabeçalho, cartão, anúncio), o parâmetro `contentType` ajuda a lista a reaproveitar composições entre itens do mesmo tipo.

7.6 Grades

```
LazyVerticalGrid(
  columns = GridCells.Adaptive(minSize = 128.dp),
  horizontalArrangement = Arrangement.spacedBy(8.dp),
  verticalArrangement = Arrangement.spacedBy(8.dp),
) {
  items(tarefas, key = { it.id }) { tarefa -> CartaoTarefa(tarefa, onAlternar = { }) }
}
```

`GridCells.Adaptive(128.dp)` cabe o máximo de colunas com pelo menos 128dp e distribui a sobra entre elas. Conferido: numa largura de 400dp (sem espaçamento), a grade criou 3 colunas de 134dp, e o quarto item foi para a segunda linha. `GridCells.Fixed(2)` fixa o número de colunas. Grades adaptativas são uma forma simples de a mesma tela servir a telefone e tablet, tema de 21/09.

7.7 Controlar a rolagem

```
val estadoDaLista = rememberLazyListState()
val escopo = rememberCoroutineScope()

LazyColumn(state = estadoDaLista) { /* ... */ }

Button(onClick = { escopo.launch { estadoDaLista.animateScrollToItem(0) } }) {
  Text("Voltar ao topo")
}
```

`rememberLazyListState()` expõe e controla a rolagem (`firstVisibleItemIndex`, `scrollToItem`, `animateScrollToItem`). Rolar é uma operação que suspende, por isso a chamada vai numa corrotina iniciada pelo evento. A documentação de boas práticas recomenda `derivedStateOf` para estados calculados a partir da rolagem, como "mostrar o botão só depois do primeiro item", para não recompor a cada pixel rolado.

📖 Ref. Android Developers — Compose performance best practices: https://developer.android.com/develop/ui/compose/performance/bestpractices

7.8 Desempenho e armadilhas

Situação	O que acontece	O que fazer
`LazyColumn` dentro de `Column(Modifier.verticalScroll(...))`	exceção (abaixo)	uma só `LazyColumn` com `item { }` para o resto
itens com altura 0 até carregar (imagem assíncrona)	a lista compõe todos de uma vez e depois descarta	reservar a altura final (placeholder)
vários elementos num único `item { }`	viram uma unidade; não são compostos separadamente	um elemento por item (divisores podem ir junto)
lista sem `key` que muda	estado e rolagem trocam de item	`key = { it.id }`
ordenar ou filtrar no corpo da tela a cada recomposição	trabalho repetido	`remember(lista, filtro) { ... }`, como o MUSI faz

A exceção do aninhamento, conferida:

> `IllegalStateException: Vertically scrollable component was measured with an infinity maximum height constraints, which is disallowed. One of the common reasons is nesting layouts like LazyColumn and Column(Modifier.verticalScroll()).`

Uma rolável dentro de outra na mesma direção só funciona se a interna tiver altura fixa. Direções diferentes (`LazyRow` dentro de `LazyColumn`) são permitidas.

📖 Ref. Android Developers — Tips on using Lazy layouts: https://developer.android.com/develop/ui/compose/lists#lazy-layouts-tips

7.9 Erros comuns com listas

> Erro comum: usar o índice como `key`. `key = { indice }` não é estável: ao remover o primeiro item, todos os índices mudam, e o problema da seção 7.3 volta.

> Erro comum: ids repetidos. Gerar id com `lista.size + 1` repete ids depois de uma remoção, e a tela quebra com `Key ... was already used`. Use um contador que só cresce (como o `proximoId` do exemplo) ou ids vindos da fonte de dados.

> Erro comum: confundir `tarefa` e `it` dentro de `items`. O parâmetro da lambda de `items` é o elemento da linha; um `it` numa lambda interna (`map`, `filter`) é outra coisa. Nomeie o parâmetro externo.

> Erro comum: `Modifier.padding` na lista em vez de `contentPadding`. Os itens são cortados antes da borda ao rolar.

---

8. Formulários e validação

8.1 Duas APIs de campo de texto

O Compose tem duas formas de ligar um campo de texto ao estado:

	Baseada em valor (a do exemplo)	**Baseada em estado**
Declaração	`value = texto, onValueChange = { texto = it }`	`state = rememberTextFieldState()`
Quem guarda o texto	você, num `mutableStateOf`	um `TextFieldState` (texto, seleção e composição)
Limitar ou filtrar entrada	lógica dentro de `onValueChange`	`inputTransformation`
Formatar exibição	`visualTransformation`	`outputTransformation`
Linhas	`singleLine`, `maxLines`	`lineLimits = TextFieldLineLimits.SingleLine`
Ação do teclado	`keyboardActions = KeyboardActions(onDone = { })`	`onKeyboardAction = { }`

A documentação do Android recomenda a API baseada em estado, por gerenciar o fluxo de entrada de forma mais completa e confiável. A baseada em valor é a mais simples de ler e é a que a aula usa; as duas compilam e funcionam com Compose Multiplatform 1.12.0.

📖 Ref. Android Developers — Configure text fields: https://developer.android.com/develop/ui/compose/text/user-input

8.2 Campo baseado em valor

O formulário do exemplo:

```
OutlinedTextField(
  value = texto,
  onValueChange = onTextoChange,
  label = { Text("Nova tarefa") },
  isError = texto.isNotEmpty() && !valido,
  modifier = Modifier.weight(1f),
)
```

O campo mostra `value`; a cada tecla, chama `onValueChange` com o texto que deveria passar a ter. Como é você quem atribui, dá para filtrar:

```
onValueChange = { novo -> if (novo.length <= 25) texto = novo }
```

Um cuidado conferido: o filtro recebe a mudança inteira. Colar 30 caracteres de uma vez não guardou os 25 primeiros; rejeitou tudo, e o campo ficou vazio. Se a intenção é cortar, use `texto = novo.take(25)`.

8.3 Campo baseado em estado

```
val titulo = rememberTextFieldState()

OutlinedTextField(
  state = titulo,
  label = { Text("Nova tarefa") },
  lineLimits = TextFieldLineLimits.SingleLine,
  inputTransformation = InputTransformation.maxLength(40),
  keyboardOptions = KeyboardOptions(imeAction = ImeAction.Done),
  onKeyboardAction = { adicionar(titulo.text.toString()) },
)
```

- `rememberTextFieldState()` cria e lembra o estado, com salvamento e restauração embutidos.
- `titulo.text` é um `CharSequence`; use `toString()` para obter a `String`.
- Para alterar o texto por código (limpar depois de adicionar), use os métodos de edição: `titulo.edit { replace(0, length, "") }` ou `titulo.clearText()`.
- `InputTransformation.maxLength(n)` filtra antes de o texto entrar no estado.

Conferido com `maxLength(10)`: digitando um caractere por vez, o campo parou em `abcdefghij`; colando 15 caracteres de uma vez, a mudança inteira foi recusada e o campo continuou vazio. A ação de teclado recebeu o texto atual, e `edit { replace(...) }` limpou o campo.

A documentação avisa que um valor inicial diferente passado a `rememberTextFieldState` numa recomposição posterior não altera o estado; para mudar depois de criado, use os métodos de edição.

📖 Ref. Android Developers — Migrate to state-based text fields: https://developer.android.com/develop/ui/compose/text/migrate-state-based

8.4 Validação derivada do estado

Validação é uma função do estado, calculada a cada recomposição (seção 4.7), não um segundo estado:

```
var texto by remember { mutableStateOf("") }
var tocado by remember { mutableStateOf(false) }

val erro: String? = when {
  texto.isBlank() -> "Informe um título"
  texto.length > 20 -> "Máximo de 20 caracteres"
  else -> null
}

OutlinedTextField(
  value = texto,
  onValueChange = { texto = it; tocado = true },
  label = { Text("Título") },
  isError = tocado && erro != null,
  supportingText = { if (tocado && erro != null) Text(erro) },
  singleLine = true,
)
Button(onClick = { /* salvar */ }, enabled = erro == null) { Text("Salvar") }
```

Três decisões nesse código:

- `erro` concentra as regras e a mensagem. O botão, o `isError` e o texto de apoio leem a mesma fonte, e não há como discordarem.
- `tocado` evita mostrar erro antes de o usuário interagir. Conferido: ao abrir, o campo vazio não aparece como erro nem mostra mensagem; depois de digitar e apagar, aparece.
- `supportingText` mostra a mensagem abaixo do campo. Só a cor vermelha não diz ao usuário o que corrigir.

`isError = true` também marca o campo como inválido nas informações de acessibilidade (no teste, a propriedade de erro do nó passou a valer `Invalid input`). Leitores de tela e testes de interface enxergam essa marca.

O exemplo usa uma regra mais simples, `isError = texto.isNotEmpty() && !valido`: campo vazio não é erro; só espaços em branco é. Conferido na `App()`: o botão fica desabilitado com o campo vazio e com espaços, e habilita com texto.

Um formulário com vários campos segue o mesmo desenho, com um `erro` por campo e o botão habilitado só quando todos são `null`.

8.5 Teclado: tipo, ação e capitalização

`KeyboardOptions` pede ao teclado virtual um comportamento:

```
keyboardOptions = KeyboardOptions(
  keyboardType = KeyboardType.Email,              // teclado com @
  imeAction = ImeAction.Next,                     // tecla de ação: próximo campo
  capitalization = KeyboardCapitalization.Sentences,
)
```

Parâmetro	Valores comuns	Efeito
`keyboardType`	`Text`, `Number`, `Email`, `Phone`, `Password`	layout do teclado
`imeAction`	`Done`, `Next`, `Search`, `Send`	ícone e significado da tecla de ação
`capitalization`	`Sentences`, `Words`, `Characters`	maiúsculas automáticas

A ação é tratada em `keyboardActions = KeyboardActions(onDone = { ... })` (baseada em valor) ou `onKeyboardAction = { ... }` (baseada em estado); as duas foram conferidas. A documentação lembra que alguns teclados podem ignorar as opções, então a validação continua necessária: `Number` não impede que chegue um texto não numérico.

📖 Ref. Android Developers — Set keyboard options:
https://developer.android.com/develop/ui/compose/text/user-input#keyboard-options

8.6 O fluxo completo de "adicionar"

```
onAdicionar = {
  tarefas = tarefas + Tarefa(proximoId, texto.trim())   // 1. cria com o texto limpo
  proximoId++                                            // 2. próximo id
  texto = ""                                             // 3. limpa o campo
}
```

- `trim()` remove espaços nas pontas; a validação usou `isNotBlank()`, coerente com isso.
- O campo é limpo alterando o estado, não o campo. O componente só reflete.
- A tarefa é criada no evento, não no corpo da composable (seção 1.4).

8.7 Erros comuns com formulários

> Erro comum: guardar `valido` num estado separado. Atualizado em `onValueChange`, ele diverge quando o texto muda por outro caminho (limpar depois de salvar). Calcule a partir do texto.

> Erro comum: erro vermelho antes de o usuário digitar. Um formulário que abre todo vermelho parece quebrado. Use um estado `tocado` ou valide ao sair do campo.

> Erro comum: filtro que recusa colagens. `if (novo.length <= n) texto = novo` rejeita a colagem inteira. Use `take(n)` ou `InputTransformation.maxLength` sabendo que ele também recusa a mudança que excede.

> Erro comum: confiar no tipo de teclado como validação. `KeyboardType.Number` é uma sugestão ao teclado, não uma garantia (e no Desktop não existe teclado virtual). Valide o conteúdo.

> Erro comum: mensagem de erro só na cor. Use `supportingText` com o texto do problema.

---

9. Material 3 e tema

9.1 O que o tema define

`MaterialTheme` fornece três sistemas que os componentes do Material 3 leem automaticamente:

```
MaterialTheme(
  colorScheme = /* cores por papel */,
  typography = /* escala de texto */,
  shapes = /* cantos arredondados */,
) {
  App()
}
```

Mudar o tema muda `Button`, `Card`, `OutlinedTextField` e todos os outros, sem tocar neles. O contrário também vale: um componente que fixa `Color(0xFF6200EE)` ignora o tema e não acompanha o modo escuro.

📖 Ref. Android Developers — Material Design 3 in Compose: https://developer.android.com/develop/ui/compose/designsystems/material3

9.2 Cores por papel

O `ColorScheme` não tem "azul" e "cinza"; tem papéis. Os mais usados:

Papel	Uso	Par de conteúdo
`primary`	ações principais, estados ativos	`onPrimary`
`primaryContainer`	fundos de destaque (item selecionado)	`onPrimaryContainer`
`secondary`, `tertiary`	ações e destaques secundários	`onSecondary`, `onTertiary`
`surface`	fundo de telas e cartões	`onSurface`
`surfaceVariant`	fundos alternativos	`onSurfaceVariant` (textos de apoio)
`error`	estados de erro	`onError`
`outline`	bordas	—

Cada cor de fundo tem um par "on" para o conteúdo por cima, com contraste garantido pelo esquema. Use sempre os pares: texto em `primaryContainer` usa `onPrimaryContainer`. O MUSI usa `MaterialTheme.colorScheme.onSurfaceVariant` para o texto secundário do `CartaoObra`.

```
Text(
  "${obra.artista} · ${obra.ano}",
  style = MaterialTheme.typography.bodyMedium,
  color = MaterialTheme.colorScheme.onSurfaceVariant,
)
```

📖 Ref. Material Design 3 — Color roles: https://m3.material.io/styles/color/roles

9.3 Claro e escuro

```
MaterialTheme(
  colorScheme = if (isSystemInDarkTheme()) darkColorScheme() else lightColorScheme()
) {
  App()
}
```

`isSystemInDarkTheme()` lê a preferência do sistema; os construtores `lightColorScheme()` e `darkColorScheme()` têm valores padrão para todos os papéis e aceitam só as cores que você quiser trocar. Para um esquema com as cores do projeto, o Material Theme Builder gera o código a partir de uma cor de origem.

A `App()` do exemplo já tem um `MaterialTheme { }` sem parâmetros dentro dela. Envolvê-la com o tema escuro, como no passo 5, funciona: os parâmetros padrão de `MaterialTheme` repetem os valores do tema de fora. Conferido: um `MaterialTheme { }` interno, dentro de um tema com `darkColorScheme()`, continuou com as cores escuras.

Um detalhe que só aparece ao testar o modo escuro: o tema não pinta o fundo sozinho, nem define a cor do texto solto. A cor de conteúdo (`LocalContentColor`) é definida por componentes como `Surface` e `Scaffold`. Conferido: dentro de `Surface(color = primary)`, a cor de conteúdo era exatamente `onPrimary`; fora de qualquer `Surface`, era preto. Na `App()` do exemplo, que não usa `Surface` nem `Scaffold`, o título "Minhas tarefas" fica com a cor padrão, e o fundo é o da janela (no Android, o tema claro declarado no manifesto). Cartões e botões mudam, porque têm cores próprias; a tela como um todo não fica escura. O MUSI evita isso envolvendo a tela num `Surface`:

```
MaterialTheme(colorScheme = esquema) {
  Surface(Modifier.fillMaxSize()) {     // pinta o fundo com 'surface' e define 'onSurface' para o
        conteúdo
    App()
  }
}
```

9.4 Tipografia e formas

A escala de texto do Material 3 tem cinco grupos, cada um em três tamanhos:

Grupo	Tamanhos	Uso típico
`display`	Large, Medium, Small	números e títulos grandes, raros
`headline`	L, M, S	título de tela (`headlineSmall` no exemplo, 24sp)
`title`	L, M, S	título de cartão e seção
`body`	L, M, S	texto corrido
`label`	L, M, S	botões, rótulos, legendas

```
Text("Minhas tarefas", style = MaterialTheme.typography.headlineSmall)
```

Conferido: `headlineSmall` tem 24sp, como na tabela da documentação. Para mudar a fonte do app, cria-se um `Typography(...)` com os estilos alterados e passa-se ao `MaterialTheme`. `shapes` define os cantos (`extraSmall` a `extraLarge`) usados pelos componentes.

Use os estilos do tema em vez de `fontSize = 18.sp` espalhado pelo código: a hierarquia de texto fica consistente e muda num lugar só.

9.5 Cor dinâmica (Android 12+)

No Android 12 (API 31) em diante, o sistema pode derivar um esquema de cores do papel de parede do usuário. As funções `dynamicLightColorScheme` e `dynamicDarkColorScheme` recebem um `Context` Android, então só existem em `androidMain`; em `commonMain`, a referência não compila (`Unresolved reference 'dynamicLightColorScheme'`, conferido).

Com `expect` / `actual` (seção 2.2), o tema comum pede o esquema a cada plataforma:

```
// commonMain
@Composable
expect fun esquemaDeCores(escuro: Boolean): ColorScheme

@Composable
fun TemaTarefas(conteudo: @Composable () -> Unit) {
  MaterialTheme(colorScheme = esquemaDeCores(isSystemInDarkTheme()), content = conteudo)
}
```

```
// androidMain
@Composable
actual fun esquemaDeCores(escuro: Boolean): ColorScheme {
  val contexto = LocalContext.current
  return when {
    Build.VERSION.SDK_INT >= Build.VERSION_CODES.S && escuro -> dynamicDarkColorScheme(contexto)
    Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> dynamicLightColorScheme(contexto)
    escuro -> darkColorScheme()
    else -> lightColorScheme()
  }
}

// desktopMain
@Composable
actual fun esquemaDeCores(escuro: Boolean): ColorScheme =
  if (escuro) darkColorScheme() else lightColorScheme()
```

O código compilou nos dois alvos. Com `minSdk = 24`, a verificação de versão é obrigatória: abaixo do Android 12, cai no esquema fixo.

📖 Ref. Android Developers — Dynamic color schemes: https://developer.android.com/develop/ui/compose/designsystems/material3#dynamic_color_schemes

9.6 Componentes que aparecem no curso

Componente	Para quê	Observação
`Button`, `FilledTonalButton`, `OutlinedButton`, `TextButton`	ações, da mais à menos enfática	uma ação principal por tela
`Card`, `ElevatedCard`, `OutlinedCard`	agrupar informações de um item	`CartaoTarefa`, `CartaoObra`
`Checkbox`, `Switch`	ligar e desligar	`Checkbox` para itens de lista; `Switch` para configurações
`OutlinedTextField`, `TextField`	entrada de texto	capítulo 8
`FilterChip`	filtros selecionáveis	`FiltrosRapidos` do MUSI
`Surface`	fundo com cor e cor de conteúdo	raiz da tela (seção 9.3)
`Scaffold`, `TopAppBar`, `FloatingActionButton`	estrutura de tela	`TopAppBar` exige `@OptIn` (seção 6.6)

9.7 Erros comuns com tema

> Erro comum: cores fixas. `Color.Black` para texto e `Color.White` para fundo quebram no modo escuro. Use os papéis do `colorScheme`.

> Erro comum: testar o modo escuro sem `Surface` na raiz. O fundo e o texto solto não mudam (seção 9.3), e parece que o tema "não pegou".

> Erro comum: par de cores trocado. Texto `onSurface` sobre `primary` pode ficar ilegível. Cada fundo usa o seu "on".

> Erro comum: cor dinâmica em `commonMain`. Não compila. Use `expect` / `actual` e verifique a versão do Android.

> Erro comum: tamanho de fonte fixo em vez de estilo. `fontSize = 13.sp` em vários lugares cria uma escala própria e inconsistente. Use `MaterialTheme.typography`.

---

10. `@Preview` a fundo

A seção 2.6 cobriu a configuração do Preview no projeto KMP. Este capítulo trata de como usá-lo para ver vários estados de uma tela sem rodar o app. Todas as anotações e parâmetros mostrados abaixo compilaram em `commonMain`, com a anotação `androidx.compose.ui.tooling.preview.Preview`. A renderização acontece no Android Studio.

10.1 Parâmetros do `@Preview`

```
@Preview(
  name = "Cartão concluído",
  group = "cartões",
  widthDp = 360,
  heightDp = 120,
  showBackground = true,
  backgroundColor = 0xFFEEEEEE,
  locale = "pt-rBR",
  fontScale = 1.5f,
  uiMode = 0x21,                // noite (em androidMain: Configuration.UI_MODE_NIGHT_YES or
        UI_MODE_TYPE_NORMAL)
  device = "id:pixel_5",
)
@Composable
fun CartaoConcluidoPreview() {
  TemaTarefas {
    CartaoTarefa(Tarefa(1, "Estudar Compose", feita = true), onAlternar = {})
  }
}
```

Parâmetro	Para quê
`name`, `group`	nome no painel; agrupar e filtrar previews
`widthDp`, `heightDp`	tamanho da área, em dp
`showBackground`, `backgroundColor`	fundo atrás do componente
`locale`	idioma dos recursos
`fontScale`	simular o tamanho de fonte do sistema
`uiMode`	modo noturno e tipo de dispositivo
`device`	dimensões e densidade de um aparelho

Em `commonMain`, as constantes `Configuration.UI_MODE_*` do Android não estão disponíveis; o valor numérico resolve (`0x21` = noite + tipo normal). A documentação lembra que esses argumentos valem só para o preview: não são aplicados quando o app roda.

Um cuidado com `uiMode`: ele muda o que `isSystemInDarkTheme()` responde. Se o preview não passa por um tema que leia essa função (como `TemaTarefas`), nada fica escuro.

📖 Ref. Android Developers — Preview your UI:
https://developer.android.com/develop/ui/compose/tooling/previews

10.2 Vários previews de uma vez

Três formas, da mais simples à mais reutilizável:

```
// 1. Várias funções
@Preview @Composable fun CartaoPendentePreview() { /* ... */ }
@Preview @Composable fun CartaoConcluidoPreview() { /* ... */ }

// 2. Modelos prontos (multipreview)
@PreviewLightDark        // claro e escuro
@PreviewFontScale        // várias escalas de fonte
@PreviewScreenSizes      // telefone, dobrável, tablet, desktop
@Composable
fun CartaoVariacoesPreview() {
  TemaTarefas { CartaoTarefa(Tarefa(1, "Estudar Compose"), onAlternar = {}) }
}

// 3. Anotação própria que reúne vários @Preview
@Preview(name = "fonte pequena", fontScale = 0.85f)
@Preview(name = "fonte grande", fontScale = 1.5f)
annotation class PreviewsDeFonte
```

As três formas compilaram em `commonMain`. Uma anotação própria, como `@PreviewsDeFonte`, vira o padrão do grupo: todo componente novo ganha o mesmo conjunto de verificações visuais.

📖 Ref. Android Developers — Multipreview templates:
https://developer.android.com/develop/ui/compose/tooling/previews#multipreview-templates

10.3 Dados de exemplo com `@PreviewParameter`

Para ver o mesmo componente com vários dados, sem uma função por caso:

```
class TarefasDeExemplo : PreviewParameterProvider<Tarefa> {
  override val values = sequenceOf(
    Tarefa(1, "Curta"),
    Tarefa(2, "Um título bem mais longo para ver como a linha quebra no cartão", feita = true),
  )
}

@Preview
@Composable
fun CartaoComDadosPreview(@PreviewParameter(TarefasDeExemplo::class) tarefa: Tarefa) {
  CartaoTarefa(tarefa, onAlternar = {})
}
```

O Android Studio gera um preview por valor da sequência. Casos que vale sempre incluir: título curto, título muito longo, lista vazia, estado de erro. São exatamente os que quebram layout na apresentação.

10.4 `LocalInspectionMode`: saber que está no preview

```
val emPreview = LocalInspectionMode.current
Text(if (emPreview) "Dados de exemplo" else carregarTitulo())
```

`LocalInspectionMode.current` vale `true` durante a renderização do preview. Serve para contornar as limitações do preview (sem rede, sem arquivos), trocando uma chamada real por dados fixos. Use com parcimônia: se a tela precisa muito disso, o problema costuma ser o componente buscar dados em vez de recebê-los por parâmetro.

📖 Ref. Android Developers — LocalInspectionMode: https://developer.android.com/develop/ui/compose/tooling/previews#localinspectionmode

10.5 Modo interativo e execução

- Modo interativo: no painel do preview, permite clicar e digitar num ambiente isolado. Estado com `remember` funciona; é uma forma rápida de conferir um formulário.
- Run Preview: instala só aquele preview no emulador ou aparelho, sem navegar pelo app.

Para o ciclo de edição da tela inteira, o Hot Reload no Desktop (seção 2.5) costuma ser mais rápido; para variações de um componente (claro, escuro, fonte, dados), o preview é imbatível.

10.6 Previews e arquitetura

A documentação do Android aponta que previews têm dificuldade com `ViewModel`: o preview não sabe construir as dependências dele. A recomendação é separar a tela que obtém dados da tela que só mostra, e fazer preview da segunda. É o mesmo desenho do capítulo 4: a `TelaAcervo` do MUSI recebe `obras`, `filtro` e `aoTrocarFiltro`, e por isso pode ser pré-visualizada com o acervo de exemplo.

```
@Composable
fun TelaDeTarefasRota(repositorio: RepositorioDeTarefas) {   // obtém dados: sem preview
  val tarefas by repositorio.tarefas.collectAsState()
  TelaDeTarefas(tarefas = tarefas, onAlternar = repositorio::alternar)
}

@Composable
fun TelaDeTarefas(tarefas: List<Tarefa>, onAlternar: (Int) -> Unit) { /* só mostra */ }

@Preview
@Composable
fun TelaDeTarefasPreview() {
  TelaDeTarefas(tarefas = listOf(Tarefa(1, "A"), Tarefa(2, "B", feita = true)), onAlternar = {})
}
```

(O `RepositorioDeTarefas` com `tarefas` observável é ilustrativo; `ViewModel` e fontes de dados são assunto da Sprint 2.)

📖 Ref. Android Developers — Previews and ViewModels: https://developer.android.com/develop/ui/compose/tooling/previews#preview-viewmodel

10.7 Erros comuns com previews

> Erro comum: preview sem tema. O componente aparece com cores padrão e não reage a `uiMode`. Envolva em `TemaTarefas` (ou no `MaterialTheme` do projeto).

> Erro comum: preview da tela que busca dados. Rede, arquivo ou `ViewModel` no caminho impedem a renderização. Faça preview da versão que recebe dados por parâmetro.

> Erro comum: um único preview com o caso feliz. Título longo, lista vazia e erro são os casos que quebram. Use `@PreviewParameter` ou vários previews.

> Erro comum: import da anotação antiga. `org.jetbrains.compose.ui.tooling.preview.Preview` não compila com a dependência do exemplo (seção 2.7).

---

11. MUSI, próxima aula, exercícios e dúvidas

11.1 Onde isto encosta no MUSI

A pasta `app/` do MUSI usa as mesmas peças, numa tela de acervo:

Conceito	No MUSI	Capítulo
tela sem estado próprio	`TelaAcervo(obras, filtro, aoTrocarFiltro)`	4
estado elevado até o ponto de entrada	`var filtro by remember { ... }` em `main.kt`	4
cálculo caro lembrado por chave	`remember(obras, filtro) { obras.filter { ... } }`	4, 7
componente com `modifier`	`CartaoObra(obra, modifier)`	6
lista com `key`	`items(visiveis, key = { it.id })`	7
lista vazia com explicação	`if (visiveis.isEmpty()) Text("Nenhuma obra...")`	7
cores e estilos do tema	`onSurfaceVariant`, `typography.titleMedium`	9
`Surface` na raiz	`MaterialTheme { Surface(Modifier.fillMaxSize()) { ... } }`	9
igualdade por valor	`FilterChip(selected = selecionado == filtro)`	3, 5
source set Desktop chamado `jvmMain`	`src/jvmMain/.../main.kt`, tarefa `:app:hotRunJvm`	2

O último item confirma a seção 2.4: o MUSI segue a nomenclatura atual (`jvmMain`); o exemplo da aula usa `desktopMain`. O conceito é o mesmo.

11.2 A rubrica da Sprint 1 e este guia

Item da rubrica	Onde está	Situação
telas do MVP com componentes próprios reutilizáveis	capítulos 4, 6 e 7	visto em 14/09
formulário com validação	capítulo 8	visto em 14/09
Material 3 com modo claro e escuro	capítulo 9	visto em 14/09
layout adaptado a pelo menos 2 larguras	seções 6.7 e 7.6 introduzem	21/09
acessibilidade (descrições, contraste, alvos de toque)	seções 8.4 e 9.2 introduzem	21/09
Navigation Compose com rotas tipadas e deep link	—	21/09
testes de interface	—	21/09

11.3 Próxima aula (21/09)

- Responsividade e adaptatividade com classes de tamanho de janela.
- Acessibilidade.

- Navegação com Navigation Compose: rotas tipadas, argumentos, pilha de retorno e deep links.
- Testes em Compose Multiplatform: `kotlin.test` e teste de interface. Os comportamentos deste guia foram conferidos com esse tipo de teste; a aula mostra como escrever os do projeto.

📖 Ref. Compose Multiplatform — Navigation: https://kotlinlang.org/docs/multiplatform/compose-navigation.html

📖 Ref. Compose Multiplatform — Testing Compose UI: https://kotlinlang.org/docs/multiplatform/compose-test.html

11.4 Perguntas de fixação

Tente responder antes de abrir a seção indicada.

1. Por que um `TextField` com `value = ""` não muda quando o usuário digita? (1.7, 4.1)
2. Em que casos um `mutableStateOf` sem `remember` parece funcionar? Por que isso é pior do que falhar logo? (4.2)
3. O que se perde e o que se mantém entre `remember` e `rememberSaveable` quando a Activity é recriada? E quando um item da `LazyColumn` sai da tela? (4.4, 7.4)
4. Por que `weight` não compila dentro de um `Box`? (3.4, 6.5)
5. Por que `var tarefas by remember { mutableStateOf(listOf(...)) }` usa `var`, se `Tarefa` só tem `val`? (5.2)
6. Depois de `tarefas[0].feita = true`, por que `tarefas = tarefas.toList()` não atualiza a tela? (5.1)
7. Por que só o cartão alterado recompõe quando o `map` gera uma lista nova? (1.3, 5.4)
8. Qual a diferença entre `clickable { }.padding(16.dp)` e `padding(16.dp).clickable { }`? (6.4)
9. O que acontece sem `key` quando o primeiro item de uma lista com estado é removido? (7.3)
10. Por que `contentPadding` e não `Modifier.padding` numa `LazyColumn`? (7.5)
11. Por que a validação deve ser calculada a partir do texto, e não guardada num estado? (4.7, 8.4)
12. Por que a tela do exemplo não fica totalmente escura no modo escuro? (9.3)

11.5 Exercícios práticos

Cada exercício tem um critério de pronto verificável na execução ou no preview.

1. Remover tarefa. Acrescente um botão de remover ao `CartaoTarefa` usando um slot de ações (seção 6.6). Pronto quando remover a primeira tarefa preservar o estado visual das demais.

2. Editar título. Um toque no título abre um campo de edição no próprio cartão; confirmar com a tecla Done salva com `copy`. Pronto quando a edição sobreviver a marcar outra tarefa como feita.

3. Seções. Mostre pendentes e concluídas na mesma `LazyColumn`, com um cabeçalho para cada seção e `key` em todos os itens. Pronto quando marcar uma tarefa a mover para a outra seção sem perder a rolagem.

4. Lista vazia. Quando não houver tarefas, mostre uma mensagem explicativa em vez de uma área em branco, como o MUSI faz. Pronto com um preview do estado vazio.

5. Validação completa. Título obrigatório, máximo de 60 caracteres, sem duplicar título existente (ignorando maiúsculas). Mensagem em `supportingText`, erro só depois de interagir. Pronto quando o botão só habilitar com título válido.

6. Campo baseado em estado. Reescreva o formulário com `rememberTextFieldState` e `InputTransformation.maxLength(60)`. Pronto quando colar um texto longo tiver um comportamento que o grupo decidiu e documentou.

7. Tema do projeto. Gere um esquema de cores com o Material Theme Builder, crie `TemaDoProjeto` com claro e escuro, `Surface` na raiz e cor dinâmica no Android 12+. Pronto com `@PreviewLightDark` da tela principal mostrando fundo e texto corretos nos dois modos.

8. Estado salvável. Faça o texto do formulário e a tarefa em edição sobreviverem à recriação da Activity com `rememberSaveable` (e um `Saver`, se necessário). Pronto quando nenhum tipo não salvável for guardado no `Bundle`.

9. Grade adaptativa. Troque a lista por `LazyVerticalGrid(GridCells.Adaptive(160.dp))` quando a largura permitir. Pronto quando a janela Desktop estreita mostrar uma coluna e a larga, várias.

10. Voltar ao topo. Um botão que aparece só depois de rolar e leva ao primeiro item com animação, usando `rememberLazyListState` e `derivedStateOf`. Pronto quando o botão não piscar durante a rolagem.

11. Previews de borda. Crie um `PreviewParameterProvider` com título curto, título muito longo e tarefa concluída, e uma anotação própria com escalas de fonte. Pronto quando nenhum caso quebrar o layout.

12. Ordem dos modifiers. Monte, lado a lado, `background(...).padding(...)` e `padding(...).background(...)`, e `clickable` antes e depois do `padding`. Explique no README o que cada combinação faz.

11.6 Dúvidas frequentes

- *A tela não atualiza quando mudo uma tarefa.* O objeto foi alterado por dentro (`var` na data class) ou a lista mutável recebeu `add` sem escrita no estado. Crie o valor novo com `copy` e reatribua (5.1, 5.2).
- *Meu contador volta a zero.* Falta `remember`, ou o estado é recriado quando a função recompõe (4.2).
- *O estado de um cartão foi parar em outro.* Falta `key` estável na lista (7.3).
- *Perdi o estado ao rolar a lista.* `remember` dentro do item é descartado quando ele sai da tela; use `rememberSaveable` com `key`, ou eleve o estado (7.4).
- *`IllegalStateException: ... measured with an infinity maximum height constraints`.* Há uma `LazyColumn` dentro de uma coluna rolável na mesma direção (7.8).
- *`Key "..." was already used`.* Dois itens com a mesma chave (7.3, 7.9).
- *`weight` não compila.* Só existe dentro de `Row` e `Column` (3.4).
- *`Smart cast to 'Tarefa' is impossible`.* O estado é uma propriedade delegada; use `?.let` ou um `val` local (3.6).
- *`Property delegate must have a 'getValue(...)' method`.* Faltam os imports `androidx.compose.runtime.getValue` e `setValue` (3.7).
- *`This material API is experimental`.* Componente experimental, como `TopAppBar`; acrescente `@OptIn(ExperimentalMaterial3Api::class)` (6.6).

- *O modo escuro não escurece a tela toda.* Falta `Surface` ou `Scaffold` na raiz (9.3).
- `dynamicLightColorScheme` *não é encontrado.* A função é só do Android; use `expect` / `actual` (9.5).
- *O* `@Preview` *não aparece ou não compila.* Confira o import `androidx...Preview`, as dependências e se o preview não depende de rede ou `ViewModel` (2.6, 10.6).
- *O preview noturno continua claro.* O componente não passou por um tema que leia `isSystemInDarkTheme()` (10.1).
- *Colar um texto longo apaga o que eu queria manter.* O filtro de `onValueChange` ou o `maxLength` recusam a mudança inteira; decida entre recusar e cortar (8.2, 8.3).
- *O app quebra no Android com* `Parcel: unknown type`. Um tipo próprio foi guardado com `rememberSaveable`; use um `Saver` ou guarde só o id (4.4).

Referências

Compose (Android Developers)

- Thinking in Compose: https://developer.android.com/develop/ui/compose/mental-model
- Jetpack Compose phases: https://developer.android.com/develop/ui/compose/phases
- State and Jetpack Compose: https://developer.android.com/develop/ui/compose/state
- State hoisting: https://developer.android.com/develop/ui/compose/state-hoisting
- Save UI state in Compose: https://developer.android.com/develop/ui/compose/state-saving
- Architecting your Compose UI: https://developer.android.com/develop/ui/compose/architecture
- Stability in Compose: https://developer.android.com/develop/ui/compose/performance/stability
- `SnapshotStateList`: https://developer.android.com/reference/kotlin/androidx/compose/runtime/snapshots/SnapshotStateList
- Lazy lists (listas, `items`, `key`): https://developer.android.com/develop/ui/compose/lists
- Compose modifiers: https://developer.android.com/develop/ui/compose/modifiers
- Layout basics: https://developer.android.com/develop/ui/compose/layouts/basics
- Scaffold: https://developer.android.com/develop/ui/compose/components/scaffold
- Performance best practices: https://developer.android.com/develop/ui/compose/performance/bestpractices
- Migrar para campos baseados em estado: https://developer.android.com/develop/ui/compose/text/migrate-state-based
- Campos de texto: https://developer.android.com/develop/ui/compose/text/user-input
- Material 3 no Compose: https://developer.android.com/develop/ui/compose/designsystems/material3
- Preview (`@Preview`): https://developer.android.com/develop/ui/compose/tooling/previews
- Material 3 (guia de design): https://m3.material.io
- Material 3 — Color roles: https://m3.material.io/styles/color/roles

Android

- `<activity>` e `android:configChanges`: https://developer.android.com/guide/topics/manifest/activity-element
- Android Emulator: https://developer.android.com/studio/run/emulator

Kotlin Multiplatform e Compose Multiplatform

- Criar o primeiro app (estrutura, alvos): https://kotlinlang.org/docs/multiplatform/compose-multiplatform-create-first-app.html
- Estrutura de projeto KMP: https://kotlinlang.org/docs/multiplatform/multiplatform-discover-project.html
- `expect` e `actual`: https://kotlinlang.org/docs/multiplatform/multiplatform-expect-actual.html
- Compose UI previews no KMP: https://kotlinlang.org/docs/multiplatform/compose-previews.html
- Compose Hot Reload: https://kotlinlang.org/docs/multiplatform/compose-hot-reload.html
- Compose Multiplatform e Jetpack Compose: https://kotlinlang.org/docs/multiplatform/compose-multiplatform-and-jetpack-compose.html
- Navigation (21/09): https://kotlinlang.org/docs/multiplatform/compose-navigation.html
- Testing Compose UI (21/09): https://kotlinlang.org/docs/multiplatform/compose-test.html

Kotlin

- Lambdas, trailing lambdas e lambda com receptor: https://kotlinlang.org/docs/lambdas.html
- Functions (argumentos nomeados e padrão): https://kotlinlang.org/docs/functions.html
- Data classes: https://kotlinlang.org/docs/data-classes.html
- Null safety: https://kotlinlang.org/docs/null-safety.html
- Delegated properties: https://kotlinlang.org/docs/delegated-properties.html
- Comparison to Java: https://kotlinlang.org/docs/comparison-to-java.html